LECTURES

16.10.
Model and O-notation

19.10.
Divide and conquer

22.10.
Master theorem proof

26.10.
Selection in O(n)

30.10.
Counting inversions

02.11.
Dynamic programming

06.11.
Search trees and TSP

Freie Universität Berlin
Department of Computer Science

LECTURE NOTES ▪ Winter Semester 2026/27

HA Skript

Höhere Algorithmik

Design and Analysis of Algorithms

CHAPTERS

1 Introduction and Overview

2 Divide and Conquer

3 Dynamic Programming

Sowas

Freie Universität Berlin ▪ Stand: 06.11.2026

Contents

1 Introduction and Overview 5

Lecture 16.10.2026. What we measure and on which machine: RAM model, uniform vs. logarithmic cost, worst-case complexity, Landau symbols.

1.1 Goals and Main Topics of the Class . . . 5
1.2 Computational Model . . . 6
1.3 Complexity . . . 8
1.4 Asymptotic Runtime as $n \to \infty$ . . . 9
1.5 O-Notation for Algorithms ($n \to \infty$) . . . 9

2 Divide and Conquer 11

Split, solve, combine: three ways to solve recurrences, Karatsuba multiplication, the master theorem with full proof, linear-time selection, and counting inversions.

A Foundations: Recurrences and Multiplication 11

Lecture 19.10.2026 ▪ the paradigm, three ways to solve recurrences, Karatsuba

2.1 Divide and Conquer . . . 11
2.2 Solving Divide-and-Conquer Recurrences . . . 12
2.2.1 Method 1: Recursion Tree . . . 12
2.2.2 Method 2: Iterative Expansion . . . 13
2.2.3 Method 3: Proof by Induction . . . 13
2.3 Multiplication Methods . . . 14
2.3.1 School Method: $O(n^2)$ . . . 14
2.3.2 Divide and Conquer . . . 14
2.3.3 Karatsuba Algorithm . . . 15
2.3.4 Proof by Induction for Karatsuba . . . 15

B The Master Theorem 17

Lectures 19.10. and 22.10.2026 ▪ statement, regularity, proof, shifting

2.4 The Master Theorem . . . 17
2.5 Regularity Condition . . . 18
2.6 Not Covered: the Gap Between Case II and III . . . 18
2.7 Proof of the Master Theorem . . . 19

Step 1: The arguments must shrink . . . 19
Step 2: Positivity and the base . . . 19
Step 3: Sandwich . . . 19
Step 4: Upper bound via a comparison function $\hat{T}$ . . . 20
Step 5: Removing the ceiling by a shift . . . 20
Step 6: Solving (6) . . . 21
2.8 Shifting the Domain . . . 21
2.9 Shifting the Value . . . 22

C **Selection in Linear Time** **23**
Lecture 26.10.2026 ▪ Quickselect, random pivot, median of medians

2.10 The Selection Problem . . . 23
2.11 Quickselect . . . 23
2.12 Randomized Pivot . . . 24
2.13 Deterministic Algorithm: Median of Medians . . . 24

D **Counting Inversions** **26**
Lecture 30.10.2026 ▪ extended Mergesort, weighted inversions, insertion sort, search trees

2.14 Inversions . . . 26
2.14.1 A Geometric View . . . 27
2.15 Counting Inversions with Mergesort . . . 27
2.15.1 Counting While Merging . . . 27
2.15.2 Embedding into Mergesort . . . 28
2.16 Variation: Weighted Inversions . . . 29
2.17 Inversions and Insertion Sort . . . 29
2.18 Counting with a Search Tree . . . 30
2.18.1 Example . . . 30
2.18.2 Rank Queries . . . 31
2.18.3 Backwards: Removing Instead of Inserting . . . 31

E **Overview** **32**
All algorithms of the chapter at a glance

2.19 Summary of Divide and Conquer . . . 32
Notes on Corrections . . . 33

3 Dynamic Programming **35**

Overlapping subproblems solved once and stored in a table: knapsack, backtracking, pseudopolynomial time, memoization, saving space, optimal search trees, polygon triangulation, traveling

salesman.

A The DP Method on Knapsack **35**

Lecture 02.11.2026 ▪ subproblems, recursion, table, backtracking, analysis

3.1 The Knapsack Problem . . . 35

3.2 The Dynamic Programming Recipe . . . 36

3.2.1 Step 1: Subproblems . . . 36

3.2.2 Step 2: The Recursion . . . 36

3.2.3 Step 3: The DP Table . . . 36

3.2.4 Recovering the Solution . . . 37

3.2.5 DP as a Path Problem . . . 38

3.2.6 Analysis . . . 38

B Variations and Implementation **38**

Lecture 02.11.2026 ▪ swapped roles, memoization, saving space

3.3 Variation: Minimum Weight for a Given Value . . . 38

3.4 Top-Down: Memoization . . . 39

3.5 Saving Space . . . 40

C Interval DP: Search Trees and Triangulations **41**

Lecture 06.11.2026 ▪ optimal binary search trees, minimum weight triangulation

3.6 Optimal Binary Search Trees . . . 41

3.6.1 The Problem . . . 41

3.6.2 Step 1: Subproblems . . . 42

3.6.3 Step 2: The Recursion . . . 43

3.6.4 Step 3: The DP Table . . . 43

3.7 Polygon Triangulation . . . 44

D DP over Subsets: Traveling Salesman **45**

Lecture 06.11.2026 ▪ Hamiltonian cycles, Held–Karp in $O(2^n n^2)$

3.8 The Traveling Salesman Problem . . . 45

Notes on Corrections . . . 47

Key definitions and results

Def. 1.1	RAM: registers, indirect addressing, jumps	6
Def. 1.2	Uniform and logarithmic cost model	7
Def. 1.3	Worst-case complexity $T(n)$, $S(n)$	8
Def. 1.5	Landau symbols $O, o, \Omega, \omega, \Theta$	9
Lemma 1.6	Limit criterion for o, Θ, ω	10
Thm. 2.2	Master theorem: $T(n) = aT(n/b) + f(n)$, three cases	17
Thm. 2.3	Median of medians: selection in $O(n)$	25
Def. 2.4	Inversion, measure of sortedness	26
Thm. 2.5	Counting (weighted) inversions in $O(n \log n)$	28
Def. 3.1	Knapsack problem	35
Thm. 3.3	Knapsack by DP in $O(nB)$, pseudopolynomial	38
Thm. 3.7	Optimal binary search tree in $O(n^3)$	44
Thm. 3.11	TSP by DP (Held–Karp) in $O(2^n n^2)$	46

Chapter 1

Introduction and Overview

Lecture of 16.10.2026 ▪ RAM model, cost models, complexity, O-notation

Before we design and compare algorithms, we fix *what* we measure and *on which machine*. This chapter introduces the RAM as computational model, the cost models, time and space complexity, and the O-notation, which compares runtimes independently of constants and small inputs.

1.1 Goals and Main Topics of the Class

The course has two sides. On the **design** side we learn general principles that work for many problems at once. On the **analysis** side we prove that an algorithm is correct and determine the resources it needs.

General design principles	Analysis
Divide and conquer	Correctness
Dynamic programming	Runtime
Greedy method	Space
Branch and bound	Lower bounds, NP-hardness / NP-completeness, reductions

- **Divide and conquer** splits a problem into smaller subproblems, solves them recursively and combines the partial solutions.
- **Dynamic programming** solves overlapping subproblems only once and stores their results in a table.
- **Greedy methods** build a solution step by step and always take the choice that looks best at the moment.
- **Branch and bound** searches all solutions systematically, but cuts off branches that provably cannot beat the best solution found so far.

All four principles are typically used to solve **optimization problems**.

The analysis also covers negative results. Sometimes no efficient algorithm is likely to exist (NP-hardness, NP-completeness), or every algorithm needs at least a certain time (lower bounds). The main tool is the **reduction**: it transforms every instance of a problem A into an instance of a problem B, so an efficient algorithm for B would also solve A. If A is hard, B must be hard as well.

1.2 Computational Model

"The runtime of an algorithm" only makes sense once we fix the machine. Candidates:

- **Actual machine** – too dependent on hardware, compiler and technology; results are not comparable.
- **Turing machine** – mathematically precise, but primitive: it moves step by step along a tape, so simple operations cost many steps.
- **RAM** (random-access machine) – realistic: it accesses any memory cell directly, like a real computer.

The RAM combines both advantages: it is precisely defined and close to real computers.

The RAM

Definition 1.1 (RAM). Storage: an unbounded sequence of registers $R_0, R_1, R_2, \ldots$; each register holds an integer. Commands:

- Arithmetic: $R_3 \leftarrow R_1$, $R_7 \leftarrow 15$, $R_2 \leftarrow R_2 + R_5$, with $+, -, \cdot, /$ (integer division $\lfloor x \rfloor$).
- Indirect addressing: $R_5 \leftarrow (R_0)$. If $R_0 = 100$, this copies the content of R_{100}.
- Unconditional jumps `GOTO B`; conditional jumps `IF` … `GOTO B` (B is a label).
- Stop the machine: `HALT`.

Indirect addressing is the key feature: a register's content is used as an *address*. That is exactly a **pointer**, and it lets us access $A[i]$ when i is only known at runtime. Jumps give loops and case distinctions.

Remark (input convention). $R_0 = n$ = size of the input; $R_1, \ldots, R_n$ = the input itself.

Example (pointer).

```
      R5 ← 0
      R0 ← 10
A:    R5 ← R5 + (R0)
      R0 ← R0 + 1
      IF R0 < 20 GOTO A
```

This computes the sum $R_{10} + R_{11} + \cdots + R_{19}$ in R_5. R_0 runs as a pointer through the addresses 10 to 19; after ten passes $R_0 = 20$ and the loop stops.

Bounding the Register Contents

The primitive RAM is unrealistic: registers hold *arbitrarily large* integers, there are infinitely many registers, and every command costs the same. It is even **more powerful** than a real processor, which works with words of fixed length (e.g. 64 bits, -2^{63} to 2^{63}) and finite memory. Two fixes:

1) Word-RAM. Every register holds an integer with b bits: $-2^b < x < 2^b$. With indirect addressing a program can therefore address only 2^b registers.

Assumption: $b = c \cdot \log_2 n$ for a constant c, where n is the input size.

$$\Rightarrow\ 2^b = 2^{c \log_2 n} = n^c \text{ addressable registers (polynomially many)}$$

The word length grows only logarithmically with the input. A register must at least hold an address of the input (about $\log_2 n$ bits), but words stay small enough that one operation can cost one unit. $\rightarrow$ **Uniform cost model.**

2) Unbounded integers. Registers may hold arbitrarily large integers, but large numbers become more *expensive*.

Definition 1.2 (Cost models).

- **Uniform cost model** (Word-RAM): every step costs 1 unit.
- **Logarithmic cost model** (unbounded integers): an operation costs the sum of the logarithms of all involved addresses and operands, with

$$\ell(x) = \lceil \log_2 \max(2, |x|) \rceil .$$

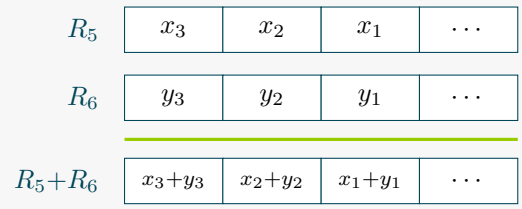
$\ell(x)$ is essentially the number of bits of x. The $\max(2, |x|)$ makes every number cost at least 1, since $\log_2 0$ is undefined and $\log_2 1 = 0$.

Example. $R_0 = 100$, $R_{100} = -77$, $R_5 = 64$. Operation $R_5 \leftarrow (R_0) + R_5$, new content of R_5 is -13.

$$\text{cost} = \ell(5) + \ell(0) + \ell(100) + \ell(-77) + \ell(5) + \ell(64) + \ell(-13)$$

Each term is one number the operation touches: $\ell(5)$ the target address; $\ell(0)$ the address of R_0 (counts as $\log_2 2$); $\ell(100)$ the content of R_0, used as address; $\ell(-77)$ the operand in R_{100} (absolute value); $\ell(5)$ and $\ell(64)$ address and content of the second operand R_5; $\ell(-13)$ the result.

Remark (cheating). Without 1) or 2), several numbers $x_1, x_2, \ldots$ could be packed into one register R_5 and $y_1, y_2, \ldots$ into R_6. One addition $R_5 + R_6$ would then compute all sums $x_i + y_i$ at once, if enough space keeps carries apart. Both the word length bound and logarithmic costs prevent this.



In this course we mostly use the **uniform cost model** (Word-RAM).

Example (Gaussian elimination). Solve an $n \times n$ system $Ax = b$ with integers of at most m bits.

- $O(n^3)$ steps in the uniform cost model.
- Rational arithmetic: $\frac{a}{b} + \frac{c}{d} = \frac{ad+cb}{bd}$.
- Logarithmic cost model – important: **reduce** the fractions. Then all numbers have $O(n(m + \log n))$ bits; without reducing, their length can grow exponentially in n.
- $\Rightarrow$ Runtime in the logarithmic cost model:

$$O(n^3 \cdot n(m + \log n)) = O(n^4(m + \log n))$$

Reduced, all intermediate values are quotients of subdeterminants of A, which is why their length stays bounded. The gcd steps for reducing are not counted separately here.

1.3 Complexity

Runtime T = number of steps; **space** S = number of registers used. Both depend on the input size n, possibly on several parameters $n, m, \ldots$ (e.g. a graph with n vertices and m edges).

Definition 1.3 (Worst-case complexity – typical for analysis).

- $T(n)$ = maximal runtime over all inputs of size $\leq n$
- $S(n)$ = the same for space

The worst case gives a **guarantee**: no input of size at most n needs more than $T(n)$ steps. Taking all inputs of size *at most* n makes T monotonically increasing.

Another possible analysis is the **average-case complexity** – only mentioned. It needs a probability distribution over the inputs, which is often unknown in practice.

1.4 Asymptotic Runtime as $n \to \infty$

Exact runtime functions are complicated and their constants depend on the model. What matters is how the runtime *grows* for large inputs.

$T(n)$	$n=1$	$n=10$	$n=100$	Class
$3n^2+1000$	1003	1300	**31 000**	$\Theta(n^2)$
n^3+20	21	**1021**	1 000 020	$\Theta(n^3)$
$2\cdot 2^n$	**4**	2048	$2\cdot 2^{100}$	$\Theta(2^n)$

Bold = winner for that n. For very small n even $2\cdot 2^n$ wins; at $n=10$, n^3+20 wins. From $n=12$ on, $3n^2+1000$ is smaller and the gap keeps growing. For $n\to\infty$ the winner is $\Theta(n^2)$: it is **asymptotically preferred**, which is not necessarily the best choice in a practical application.

1.5 O-Notation for Algorithms ($n \to \infty$)

Let $f, g, h, \cdots : \mathbb{N}_0 \to \mathbb{R}$. Only large n matter, so we require positivity only from some point on.

Definition 1.4 (almost everywhere positive, AEP). f is AEP if there is an n_0 with $f(n) > 0$ for all $n \geq n_0$.

Example: $\log_2(\log_2 n)$ is undefined for $n \leq 1$ and 0 for $n = 2$, but positive for $n \geq 3$, so it is AEP with $n_0 = 3$.

For AEP functions we write $f \leq_{\text{AE}} g$ if there is an n_0 with $f(n) \leq g(n)$ for all $n \geq n_0$ – "f is at most g, except for finitely many n".

Definition 1.5 (Landau symbols).

$$
\begin{aligned}
O(f(n)) &= \{\, g \text{ AEP} \mid \exists\, c>0 : g \leq_{\text{AE}} c\cdot f \,\} && \text{upper bound}\\
o(f(n)) &= \{\, g \text{ AEP} \mid \forall\, c>0 : g \leq_{\text{AE}} c\cdot f \,\} && \text{strict upper bound}\\
\Omega(f(n)) &= \{\, g \text{ AEP} \mid \exists\, c>0 : g \geq_{\text{AE}} c\cdot f \,\} && \text{lower bound}\\
\omega(f(n)) &= \{\, g \text{ AEP} \mid \forall\, c>0 : g \geq_{\text{AE}} c\cdot f \,\} && \text{strict lower bound}\\
\Theta(f(n)) &= O(f(n)) \cap \Omega(f(n))
\end{aligned}
$$

- $g \in O(f)$: g grows *at most* as fast as f, up to a constant factor.
- $g \in o(f)$: for every $c > 0$, g eventually stays below $c \cdot f$ – g grows *strictly slower*.
- Ω and ω are the corresponding lower bounds.
- $g \in \Theta(f)$: g grows *exactly* as fast as f, up to constant factors.

Although $O(f(n))$ is a set, one writes $g(n) = O(f(n))$, e.g. $3n^2+1000 = O(n^2)$ and $4n^2 = O(n^2)$. This is a **one-way equation**, read only left to right: it does *not* follow that $3n^2+1000 = 4n^2$. Likewise $n^2 + O(n)$ means n^2 plus some function from $O(n)$.

Upper and lower bounds are two views of one relation: $g(n) = O(f(n)) \Leftrightarrow f(n) = \Omega(g(n))$, since $g \leq_{\text{AE}} c\cdot f$ is equivalent to $f \geq_{\text{AE}} \frac{1}{c}\cdot g$.

Lemma 1.6. Let f, g be AEP and $\lim_{n\to\infty} f(n)/g(n) = C$. Then

$$C \begin{cases} = 0 & \Rightarrow f(n) \in o(g(n)) \\ \in (0, \infty) & \Rightarrow f(n) \in \Theta(g(n)) \\ = \infty & \Rightarrow f(n) \in \omega(g(n)) \end{cases}$$

Examples: $(3n^2 + 1000)/n^2 \to 3$, so $3n^2 + 1000 \in \Theta(n^2)$; $n^3/2^n \to 0$, so $n^3 \in o(2^n)$. If the limit does not exist, the lemma says nothing.

Remark (Beware: Taylor formula). $f(x + h) = f(x) + h \cdot f'(x) + O(h^2)$ as $h \to 0$. In analysis, O describes the behaviour for *small* h. In this course, O always refers to $n \to \infty$ unless stated otherwise.

Chapter 2

Divide and Conquer

Lecture of 19.10.2026 ▪ Divide and conquer, solving recurrences, Karatsuba multiplication, master theorem (continued 22.10.), selection (26.10.), inversions (30.10.)

Divide and conquer is the first general design principle. Its runtime is naturally described by a **recurrence**, an equation that expresses $T(n)$ through T of smaller arguments. We learn three methods to solve recurrences, apply the principle to multiplying large numbers, and state and prove the master theorem, which solves a whole class of recurrences at once. Then we use divide and conquer to select the median in linear time and to count the inversions of a list.

A Foundations: Recurrences and Multiplication

Lecture 19.10.2026 ▪ the paradigm, three ways to solve recurrences, Karatsuba

2.1 Divide and Conquer

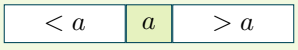
Divide and conquer is a **paradigm**: an algorithm works in three steps.

1. **Split** the problem into several smaller subproblems.
2. **Solve** the subproblems recursively.
3. **Combine** the solutions of the subproblems.

Example (sorting).

Step	Quicksort	Mergesort
1. Split	Choose a pivot a; split into lists $< a$ and $> a$: $O(n)$	Split into 2 lists of length $n/2$
2. Solve	Sort both lists recursively	Sort both lists recursively
3. Combine	Put together: $[< a \mid a \mid > a]$	Merge the two sorted lists: $O(n)$

Quicksort, step 3:



Quicksort does its work *before* the recursion, so combining is trivial. Mergesort does it *after* the recursion, so splitting is trivial. For Mergesort the runtime satisfies the recurrence

$$T(n) = 2\,T\left(\tfrac{n}{2}\right) + O(n) = O(n \log n).$$

For odd n the halves differ by one element:

$$T(n) = T\left(\lceil \tfrac{n}{2} \rceil\right) + T\left(\lfloor \tfrac{n}{2} \rfloor\right) + O(n) = O(n \log n).$$

The term $2\,T(n/2)$ stands for the two recursive calls (step 2), $O(n)$ for splitting and merging (steps 1 and 3). The factor in front is the **number of subproblems**; the check is: number of

parts × size of a part = n (here $2 \cdot n/2 = n$). The n in the formula is the size of the *current* call, so the same rule applies on every level.

Definition 2.1 (Bottom of the recursion). $T(n) \leq C_0$ for $n \leq N_0$.

Small inputs are solved directly. There are only finitely many inputs of size at most N_0, so their runtime is bounded by a constant C_0. Without this base case, T would not be determined.

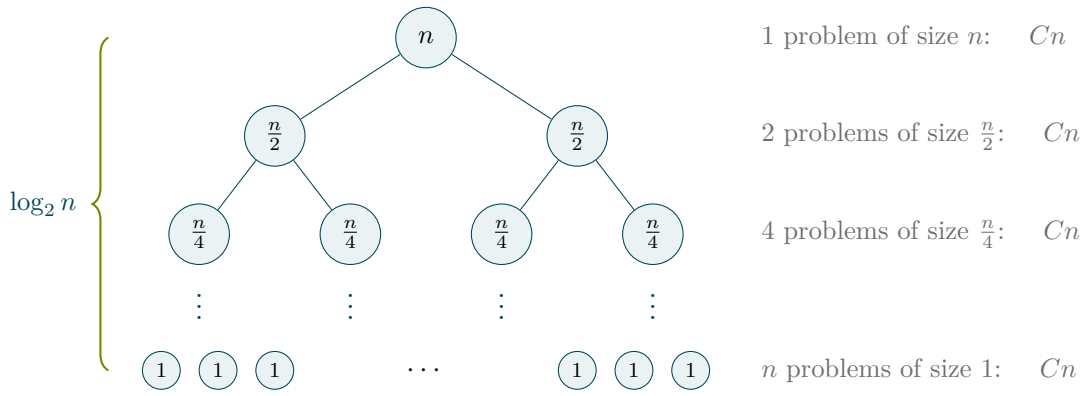
Worked example (merging costs exactly n, $T(1) = 0$): $T(2) = 2$, $T(4) = 8$, $T(8) = 24$, $T(16) = 64$ – always $n \cdot \log_2 n$.

2.2 Solving Divide-and-Conquer Recurrences

We show in three ways that $T(n) \leq 2\,T(n/2) + Cn$ gives $T(n) = O(n \log n)$. Cn replaces $O(n)$ by an explicit bound. To keep the calculations simple, $n = 2^k$.

2.2.1 Method 1: Recursion Tree

Every recursive call is a node, labelled with the work it does *itself* (without its subcalls).



Level	Problems	Size each	Work on level
0	1	n	Cn
1	2	$n/2$	Cn
2	4	$n/4$	Cn
i	2^i	$n/2^i$	Cn
$\log_2 n$	n	1	Cn

Every level costs $O(n)$. The size halves per level, so after $\log_2 n$ halvings we reach size 1.

$$\Rightarrow\ T(n) = O(n) \cdot (\log_2 n + 1) = O(n \log n)$$

2.2.2 Method 2: Iterative Expansion

Insert the recurrence into itself again and again:

$$\begin{aligned} T(n) &\le Cn + 2\,T\left(\tfrac{n}{2}\right) \\ &\le Cn + 2\left(C\tfrac{n}{2} + 2\,T\left(\tfrac{n}{4}\right)\right) \\ &\le Cn + 2\left(C\tfrac{n}{2} + 2\left(C\tfrac{n}{4} + 2\,T\left(\tfrac{n}{8}\right)\right)\right) \\ &\;\;\vdots \\ &\le Cn + 2C\tfrac{n}{2} + 4C\tfrac{n}{4} + \cdots + 2^{\log_2 n}\cdot C\cdot 1 \\ &= Cn\,(\log_2 n + 1) \end{aligned}$$

Every bracket contributes exactly Cn, one per level of the recursion tree. For the last term choose $C \ge C_0$, so $T(1) \le C \cdot 1$. The expansion is the recursion tree written as a calculation.

2.2.3 Method 3: Proof by Induction

Methods 1 and 2 *find* the solution; induction *proves* it. Given:

$$T(n) \le 2\,T\left(\tfrac{n}{2}\right) + Cn \;\text{ for } n > N_0, \qquad T(n) \le C_0 \;\text{ for } n \le N_0$$

The two lines cover separate ranges of n – a case distinction, not a contradiction. Simplifying assumption: $n = 2^k$.

> Claim. $T(n) \le A \cdot n \log_2 n$ for all $n \ge 2$, for a suitable constant A.

Induction step ($n > N_0$):

$$\begin{aligned} T(n) \le 2\,T\left(\tfrac{n}{2}\right) + Cn &\overset{\text{I.H.}}{\le} 2\left(A\cdot\tfrac{n}{2}\cdot\log_2\tfrac{n}{2}\right) + Cn \\ &= An\log_2 n - An + Cn \\ &\le An\log_2 n \qquad \text{as long as } A \ge C \end{aligned}$$

Induction base ($2 \le n \le N_0$): $T(n) \le C_0 \overset{!}{\le} A \cdot n \log_2 n$. The right side is smallest for $n = 2$: $C_0 \le 2A$, i.e. $A \ge C_0/2$.

With $\boldsymbol{A = \max(C, C_0/2)}$ both parts hold. ■

The key is the term $-An$: halving n reduces $\log_2 n$ by exactly 1, and this saving pays for the extra work Cn when $A \ge C$. The base starts at $n = 2$ because $n \log_2 n = 0$ for $n = 1$. We may assume $N_0 \ge 3$ (otherwise enlarge N_0); then $n > N_0$ implies $n/2 \ge 2$, so the induction hypothesis always applies.

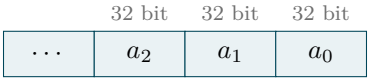
2.3 Multiplication Methods

In the logarithmic cost model, multiplying two large integers is not free: its cost grows with their length.

2.3.1 School Method: $O(n^2)$

Processors multiply 32-bit numbers directly in hardware. Larger numbers (e.g. 1000 bits) are split into 32-bit blocks; each block is one digit to the base $B = 2^{32}$.

$$A = a_0 + a_1 B + a_2 B^2 + \cdots + a_{n-1} B^{n-1}, \qquad 0 \le a_i < B$$



High-school method: form the n^2 products of every digit of x with every digit of y and add them with the correct shifts.

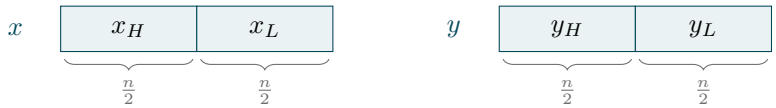
Example ($345 \cdot 986$, base 10, $n = 3$).

	$\cdot 6$	$\cdot 80$	$\cdot 900$
5	30	400	4 500
40	240	3 200	36 000
300	1 800	24 000	270 000

$n^2 = 9$ terms; their sum is $345 \cdot 986 = 340\,170$. Each digit product is one word operation, and adding the shifted results also costs $O(n^2)$.

2.3.2 Divide and Conquer

Let x, y be n-bit numbers (n even for simplicity). Split each into a high and a low half of $n/2$ bits:



$$x = x_H \cdot 2^{n/2} + x_L, \qquad y = y_H \cdot 2^{n/2} + y_L$$
$$x \cdot y = x_H y_H \cdot 2^n + (x_H y_L + x_L y_H) \cdot 2^{n/2} + x_L y_L$$

One $n \times n$ multiplication becomes **four** $(n/2) \times (n/2)$ multiplications plus additions and shifts in $O(n)$:

$$\begin{aligned} T(n) &= 4\,T\left(\tfrac{n}{2}\right) + O(n) \\ &\le Cn + 2Cn + 4Cn + \cdots + 4^k\,T(1) = Cn\,(2^k - 1) + 4^k\,T(1) = O(n^2) \end{aligned}$$

since $4^k = (2^k)^2 = n^2$. The work doubles per level, so the n^2 base cases dominate. Divide and conquer alone gives no improvement: we still compute all n^2 digit products. To gain something, we must **save multiplications**.

2.3.3 Karatsuba Algorithm

Compute the middle term without both products separately:

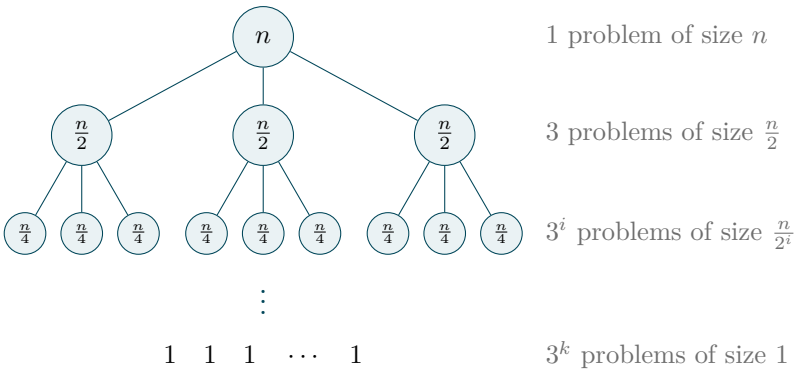
$$\begin{aligned} a &:= (x_H + x_L)(y_H + y_L) = x_H y_H + x_H y_L + x_L y_H + x_L y_L \\ b &:= x_H y_H, \qquad c := x_L y_L \\ d &:= a - b - c = x_H y_L + x_L y_H \\ \Rightarrow\ x \cdot y &= b \cdot 2^n + d \cdot 2^{n/2} + c \end{aligned}$$

Only **three** half-length multiplications (a, b, c); everything else is $O(n)$:

$$T(n) = 3\,T\left(\tfrac{n}{2}\right) + O(n)$$

> **Remark.** $x_H + x_L$ and $y_H + y_L$ can have $n/2 + 1$ bits. This extra bit does not change the asymptotic runtime.

Recursion tree $(n = 2^k)$:



Level i has 3^i problems of size $n/2^i$, with work $3^i \cdot C \cdot n/2^i = Cn \cdot (3/2)^i$. The work grows by the factor $3/2$ per level – a geometric series dominated by its last term, the 3^k base cases. With $n\,(3/2)^k = 2^k \cdot 3^k/2^k = 3^k$ and $C \geq C_0$:

$$T(n) \leq \sum_{i=0}^{k} Cn \left(\tfrac{3}{2}\right)^i \leq \frac{(3/2)^{k+1}}{3/2 - 1}\, Cn = 3Cn \left(\tfrac{3}{2}\right)^k = 3C \cdot 3^k,$$

so

$$T(n) = O(3^k), \qquad 3^k = 3^{\log_2 n} = \left(2^{\log_2 n}\right)^{\log_2 3} = n^{\log_2 3} < n^2, \quad \log_2 3 \approx 1.585$$

Saving one multiplication per level improves $O(n^2)$ to $O(n^{1.585})$.

2.3.4 Proof by Induction for Karatsuba

Recurrence $T(n) \leq 3\,T(n/2) + Cn$, with $\gamma := \log_2 3$.

First attempt: claim $T(n) \leq A \cdot n^\gamma$.

$$T(n) \leq 3\,T\left(\tfrac{n}{2}\right) + Cn \overset{\text{I.H.}}{\leq} 3\,\frac{An^\gamma}{2^\gamma} + Cn \overset{?}{\leq} An^\gamma$$

The main term needs $3A/2^\gamma \leq A$, i.e. $3 \leq 2^\gamma \Leftrightarrow \log_2 3 \leq \gamma$ – this is where the exponent comes from. But for $\gamma = \log_2 3$ exactly, $3A/2^\gamma = A$ and the term Cn is left over. The induction fails, although the claim is true.

Idea: strengthen the claim to $T(n) \le A \cdot n^\gamma - Dn$.

$$\begin{aligned} T(n) &\le 3\,T\left(\tfrac{n}{2}\right) + Cn \overset{\text{I.H.}}{\le} 3\left(A\frac{n^\gamma}{2^\gamma} - D\frac{n}{2}\right) + Cn \\ &= An^\gamma - \tfrac{3}{2}Dn + Cn = An^\gamma - Dn - \left(\tfrac{D}{2} - C\right)n \\ &\le An^\gamma - Dn \qquad \text{as long as } D \ge 2C \end{aligned}$$

For the base $1 \le n \le N_0$ choose $A \ge C_0 + D$; this works because $n^\gamma \ge n \ge 1$. A stronger claim gives a stronger induction hypothesis: the recursive calls deliver the saving $\frac{3}{2}Dn$, which pays for Cn. Hence $T(n) = O(n^{\log_2 3})$.

B **The Master Theorem**

Lectures 19.10. and 22.10.2026 ▪ statement, regularity, proof, shifting

2.4 The Master Theorem

The three methods work for every recurrence, but each time we calculate again. For divide-and-conquer recurrences there is a general result.

Theorem 2.2 (Master theorem). Let $a \geq 1$, $b > 1$, and let $f(n), T(n)$ be AEP with

$$a\,T(\lfloor \tfrac{n}{b} \rfloor) + f(n) \;\leq\; T(n) \;\leq\; a\,T(\lceil \tfrac{n}{b} \rceil) + f(n) \quad \text{for } n \geq n_0.$$

Critical exponent $\boldsymbol{\gamma := \log_b a \geq 0}$.

Case	Condition on $f(n)$	Result	Who dominates
1 ($<$)	$f(n) = O(n^{\gamma-\varepsilon})$ for some $\varepsilon > 0$	$T(n) = \Theta(n^{\gamma})$	Leaves
2 ($=$)	$f(n) = \Theta(n^{\gamma})$	$T(n) = \Theta(n^{\gamma} \log n)$	Every level
3 ($>$)	$f(n) = \Theta(n^{\gamma+\varepsilon})$ for some $\varepsilon > 0$, or f satisfies the regularity condition (Section 2.5)	$T(n) = \Theta(f(n))$	Root

- a = number of subproblems, b = factor by which the size shrinks, $f(n)$ = work for splitting and combining.
- $\lceil \cdot \rceil$ and $\lfloor \cdot \rfloor$ allow n not divisible by b (like odd n in Mergesort); rounding does not change the result.
- The recursion tree has $a^{\log_b n} = n^{\gamma}$ leaves. The theorem compares $f(n)$, the work at the root, with n^{γ}, the work at the leaves. In Case 1 the work grows per level, in Case 2 every level costs the same, in Case 3 it shrinks per level.

Remark (Case 3). The textbook formulation (e.g. CLRS) only assumes $f(n) = \Omega(n^{\gamma+\varepsilon})$ and therefore also requires the **regularity condition** $a \cdot f(n/b) \leq c \cdot f(n)$ for a constant $c < 1$ and all large n. Without it Case 3 can fail for a function f that jumps up and down. For polynomials $f(n) = n^p$ with $p > \gamma$ it holds automatically, and by Section 2.5 it even suffices on its own.

Examples.

Algorithm	a	b	γ	$f(n)$	Case	$T(n)$
Mergesort	2	2	1	$\Theta(n)$	2	$\Theta(n \log n)$
Simple D&C multiplication	4	2	2	$\Theta(n)$	1	$\Theta(n^2)$
Karatsuba	3	2	$\log_2 3 \approx 1.585$	$\Theta(n)$	1	$\Theta(n^{\log_2 3})$

For Karatsuba, Case 1 applies with $\varepsilon = \log_2 3 - 1 \approx 0.585 > 0$. All three confirm the results calculated by hand.

Lecture of 22.10.2026 ▪ Regularity condition, proof, shifting the domain and the value

Section 2.4 stated the master theorem. The following sections add the regularity condition, show a recurrence the theorem does not cover, prove it, and use the same shifting idea to bring messy

recurrences into the right form. From here on the gap is called δ, and the cases are named I ($-$), II ($=$) and III ($+$) as in the notes; they are Cases 1–3 of Theorem 2.2.

For the proof we number the two inequalities of Theorem 2.2, with $f(n), T(n) > 0$ for $n \geq n_0$:

$$(1) \quad T(n) \leq a\,T(\lceil \tfrac{n}{b} \rceil) + f(n)$$
$$(2) \quad T(n) \geq a\,T(\lfloor \tfrac{n}{b} \rfloor) + f(n)$$

2.5 Regularity Condition

Case III also holds if

$$\frac{a\,f(\lceil \frac{n}{b} \rceil)}{f(n)} \leq c < 1 \quad \text{for } n \geq n_0.$$

The work at the root then dominates by a constant factor per level, so the level costs form a decreasing geometric series. This covers non-polynomial f such as 2^n or $2^{\sqrt{n}}$.

The regularity condition alone is enough for Case III: applying $f(n) \geq \frac{a}{c} f(n/b)$ repeatedly gives $f(n) = \Omega(n^{\gamma+\delta})$ with $\delta = \log_b(1/c) > 0$. For a polynomial $f(n) = n^{\gamma+\delta}$ it holds automatically with $c = b^{-\delta} < 1$. Only with the weaker assumption $f = \Omega(n^{\gamma+\delta})$ alone must it be checked in addition (textbook form, e.g. CLRS).

Example. $f(n) = 2^n$, $a = b = 2$. With $\lceil n/2 \rceil \leq (n+1)/2$:

$$\frac{2 \cdot 2^{\lceil n/2 \rceil}}{2^n} \leq \frac{2 \cdot 2^{(n+1)/2}}{2^n} = 2^{(3-n)/2} \leq \tfrac{1}{2} \quad \text{for } n \geq 5$$

So $c = 1/2$ and $T(n) = \Theta(2^n)$.

2.6 Not Covered: the Gap Between Case II and III

$$T(n) = 2\,T(\tfrac{n}{2}) + \Theta(n \log_2 n), \qquad a = b = 2,\ \gamma = 1$$

f grows faster than $n^\gamma = n$, but not by a polynomial factor n^δ, and the regularity condition fails (the ratio tends to 1). No case applies. The recursion tree gives the answer: about $\log_2 n$ levels, each costing up to $n \log n$, so $T(n) = \Theta(n \log^2 n)$.

Remark (Extended Case II). Many textbooks close part of this gap: if $f(n) = \Theta(n^\gamma \log^k n)$ with $k \geq 0$, then $T(n) = \Theta(n^\gamma \log^{k+1} n)$. Our example is $k = 1$. For $k = -1$ one gets $\Theta(n^\gamma \log\log n)$, for $k < -1$ $\Theta(n^\gamma)$. The version from the lecture does not contain this extension.

2.7 Proof of the Master Theorem

Step 1: The arguments must shrink

We may always enlarge n_0. Then we can assume that (1) leads to a smaller argument on the right side. This is not automatic: for $b = 1.1$ and $n = 5$, $\lceil 5/1.1 \rceil = \lceil 4.55 \rceil = 5$, so $T(5)$ would depend on itself.

$$\lceil \tfrac{n}{b} \rceil < \tfrac{n}{b} + 1 \leq n \iff n \geq \frac{b}{b-1}$$

Increase n_0 if necessary so that $n_0 \geq b/(b-1)$.

Mixed recursions. Mergesort has $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + f(n)$, which is neither (1) nor (2) with equality. **Remark:** if f is monotone, $f(n+1) \geq f(n)$, this can be reduced to (1) and (2) (Step 3).

Step 2: Positivity and the base

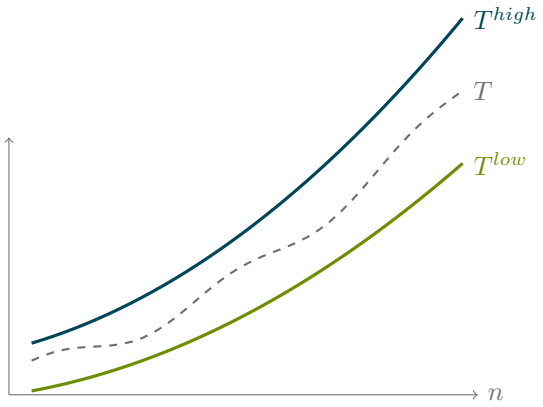
- $f(n), T(n) > 0$ for $n \geq n_0/b - 1$, by enforcing a bigger n_0 if necessary. The smallest argument $\lfloor n/b \rfloor \geq n/b - 1$ then stays in the positive range.
- (3) $T(n) \leq M$ for $n < n_0$, where M is the maximum of finitely many values.

Step 3: Sandwich

Define two clean recursions:

- $\boldsymbol{T^{high}}$ by (1) and (3) with equality,
- $\boldsymbol{T^{low}}$ by (2) with equality and $T^{low}(n) = 0$ for $n < n_0$.

> Claim. T with the mixed recursion is sandwiched: $T^{low}(n) \leq T(n) \leq T^{high}(n)$. If f is monotone, then T^{high} and T^{low} are monotone.



Proof of $T(n) \leq T^{high}(n)$ by induction (Mergesort case, $T^{high}(n) = 2T^{high}(\lceil n/2 \rceil) + f(n)$).
Base $n < n_0$: by (3), $T(n) \leq M = T^{high}(n)$.

Step:

$$\begin{aligned} T(n) &\le T(\lfloor \tfrac{n}{2} \rfloor) + T(\lceil \tfrac{n}{2} \rceil) + f(n) \\ &\overset{\text{I.H.}}{\le} T^{high}(\lfloor \tfrac{n}{2} \rfloor) + T^{high}(\lceil \tfrac{n}{2} \rceil) + f(n) \\ &\le 2\,T^{high}(\lceil \tfrac{n}{2} \rceil) + f(n) = T^{high}(n) \qquad (T^{high} \text{ monotone}) \end{aligned}$$

The lower bound $T^{low}(n) \le T(n)$ works the same way: start from $T(n) \ge T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + f(n)$, use the I.H. and $T^{low}(\lceil n/2 \rceil) \ge T^{low}(\lfloor n/2 \rfloor)$. The master theorem applies to T^{high} and T^{low}. Both give the same $\Theta(\dots)$, and T lies in between, so $T(n) = \Theta(\dots)$ as well.

Step 4: Upper bound via a comparison function $\hat{T}$

Let $f(n) = \Theta(n^\ell) \le u \cdot n^\ell$ for $n \ge n_0$, with $\ell = \gamma + \delta$, γ or $\gamma - \delta$. (Steps 4–6 treat polynomial f; for Case III with only the regularity condition, the level costs form a decreasing geometric series, Section 2.5.) **Goal:** find $\hat{T}$ with

$$\begin{aligned} &(4) \quad \hat{T}(n) \ge a\,\hat{T}(\lceil \tfrac{n}{b} \rceil) + u\,n^\ell \quad \text{for } n \ge n_0 \\ &(5) \quad \hat{T}(n) \ge M \qquad\qquad\qquad\quad \text{for } \tfrac{n_0}{b} \le n < n_0 \end{aligned}$$

> Claim. Then $T(n) \le \hat{T}(n)$.

Proof by induction. $n < n_0$ ✓ by (3) and (5). For $n \ge n_0$:

$$T(n) \le a\,T(\lceil \tfrac{n}{b} \rceil) + f(n) \overset{\text{I.H.}}{\le} a\,\hat{T}(\lceil \tfrac{n}{b} \rceil) + u\,n^\ell \overset{(4)}{\le} \hat{T}(n)$$

Step 5: Removing the ceiling by a shift

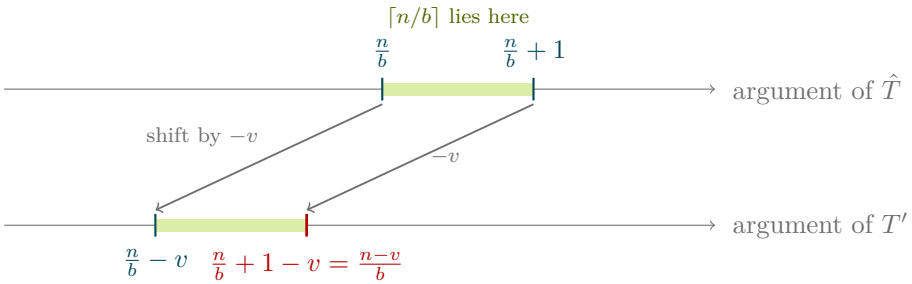
Let T' be defined on the real numbers, monotone, and a solution of

$$(6) \quad T'(x) = a\,T'(\tfrac{x}{b}) + u'\,x^\ell$$

Set $\hat{\boldsymbol{T}}(\boldsymbol{n}) := \boldsymbol{T'(n - v)}$ with $\boldsymbol{v = b/(b-1)}$. Then $v - v/b = 1$, so $(n-v)/b = n/b + 1 - v$, and

$$\begin{aligned} \hat{T}(n) = T'(n-v) &= a\,T'(\tfrac{n-v}{b}) + u'(n-v)^\ell \\ &= a\,T'(\tfrac{n}{b} + 1 - v) + u'(n-v)^\ell \\ &\ge a\,\hat{T}(\lceil \tfrac{n}{b} \rceil) + u\,n^\ell \end{aligned}$$

The last step uses $\lceil n/b \rceil \le n/b + 1$ and monotonicity of T', and $u'(n-v)^\ell \ge u\,n^\ell$ for u' big enough. So $\hat{T}$ satisfies (4).



The shift moves every possible value of $\lceil n/b \rceil$ to an argument of T' at most $\frac{n-v}{b}$, the point where (6) evaluates T'. Since T' is monotone, $\hat{T}(\lceil n/b \rceil) = T'(\lceil n/b \rceil - v) \le T'(\frac{n-v}{b})$: the ceiling no longer matters.

Step 6: Solving (6)

Idea: $T'(n) = Dn^\gamma + En^\ell$. Substitute into (6):

$$\begin{aligned} Dn^\gamma + En^\ell &\overset{!}{=} a\left(D\left(\tfrac{n}{b}\right)^\gamma + E\left(\tfrac{n}{b}\right)^\ell\right) + u'n^\ell \\ &= Dn^\gamma \underbrace{\left(a \cdot \tfrac{1}{b^\gamma}\right)}_{=1} + E\,a\,n^\ell \tfrac{1}{b^\ell} + u'n^\ell \end{aligned}$$

since $\gamma = \log_b a \Leftrightarrow b^\gamma = a$. The D-terms cancel, and the equation holds iff $\boldsymbol{E(1 - a/b^\ell) = u'}$.

Case	ℓ vs γ	b^ℓ vs a	E
$(+)$	$\ell > \gamma$	$b^\ell > a$	$E > 0$
$(-)$	$\ell < \gamma$	$b^\ell < a$	$E < 0$
$(=)$	$\ell = \gamma$	$b^\ell = a$	would need $u' = 0 \to$ no solution

- $(+)$ $T'(n) = Dn^\gamma + En^\ell$ with $E > 0$ solves (6). D is free: choose it big enough to fulfil (5). Then $T(n) \le \hat{T}(n) = T'(n - v) = O(n^\gamma) + O(n^\ell) = O(n^\ell)$.
- $(-)$ $E < 0$ solves (6), D is arbitrary. Choose D large enough to make T' monotone and to fulfil (5). Then $T(n) = O(n^\gamma) + O(n^\ell) = O(n^\gamma)$.
- $(=)$ Modified idea: $T'(n) = Dn^\gamma + En^\gamma \log_b n$. Substituting gives $Dn^\gamma + En^\gamma \log_b n - En^\gamma + u'n^\gamma$ on the right, so $E = u'$ works and $T(n) = O(n^\gamma \log n)$. (This calculation is not in the notes.)

The lower bounds follow in the same way with T^{low}.

2.8 Shifting the Domain

$$A(n) = 3\,A(\tfrac{n}{2} + 1) + 7n^2$$

The $+1$ inside the argument does not fit the master theorem. Set $B(n) := A(n - v)$ and choose v so that B has a clean recursion:

$$B(n) = A(n - v) = 3\,A(\tfrac{n-v}{2} + 1) + 7(n - v)^2 \overset{!}{=} 3\,B(\tfrac{n}{2}) + \dots$$

This needs $(n - v)/2 + 1 = n/2 - v$, i.e. $\boldsymbol{v = -2}$. So $B(n) = A(n + 2)$, and $A(n) = B(n - 2)$.

Clean recursion for $B(n)$:

$$B(n) = 3\,B(\tfrac{n}{2}) + 7(n + 2)^2$$

$\gamma = \log_2 3 \approx 1.585 < 2$ and $f = \Theta(n^2)$: Case III, so $B(n) = \Theta(n^2)$ and $A(n) = \Theta(n^2)$. It is the same trick as v in Step 5.

2.9 Shifting the Value

$$A(n) = 3\,A(\tfrac{n}{2}) + 7n^2 + 3$$

Set $C(n) := A(n) + W$:

$$C(n) = 3\,(C(\tfrac{n}{2}) - W) + 7n^2 + 3 + W = 3\,C(\tfrac{n}{2}) + 7n^2 + 3 - 2W$$

With $W = 3/2$ the constant disappears: $C(n) = 3C(n/2) + 7n^2$. In general $W = c/(a-1)$ for an additive constant c. (The choice of W is not in the notes.)

Lecture of 26.10.2026 ▪ Selection, Quickselect, median of medians

Divide and conquer also solves the **selection** problem: find the k-smallest element of a list, for example the median, in $O(n)$ instead of the $O(n \log n)$ of sorting. Quickselect with a random pivot needs expected linear time; this is a first application of the master theorem (Section 2.4) with $a = 1$. The median-of-medians pivot of Blum, Floyd, Pratt, Rivest and Tarjan (1973) makes it deterministic; its recurrence has two calls of different size, so we solve it by induction.

C Selection in Linear Time

Lecture 26.10.2026 ▪ Quickselect, random pivot, median of medians

2.10 The Selection Problem

Definition (rank, selection). An element $a \in A$ has **rank** k if it is at position k when A is sorted in increasing order. **Selecting** means: given $k \in \{1, \dots, n\}$, find the k-smallest element of A. The **median** is the element of rank $\lfloor (n+1)/2 \rfloor$.

Example. $A = 3, 4, 7, 11, 12$: the median has rank $\lfloor 6/2 \rfloor = 3$, so it is 7. For $k = 2$ we get 4.

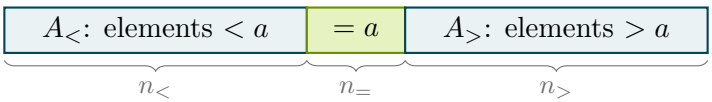
Computing the rank of a *given* element is easy: count the smaller elements, in $O(n)$. With equal elements the rank is not unique; an element can then have several possible ranks.

Goal: selection in $O(n)$. Sorting first costs $O(n \log n)$.

2.11 Quickselect

Quickselect is the selection counterpart of Quicksort: after the partition, Quicksort recurses into *both* parts, Quickselect only into the *one* that contains the answer.

Partitioning. Pick a pivot a and split A into $A_<$ (elements $< a$), $A_=$ (elements $= a$) and $A_>$ (elements $> a$):



with lengths $n_< + n_= + n_> = n$. This costs $O(n)$.

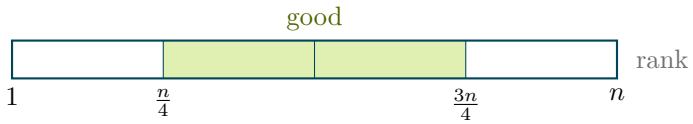
Recursion select(k, A):

1. if $k \le n_<$: return select$(k, A_<)$
2. else if $k \le n_< + n_=$: return a
3. else: return select$(k - n_< - n_=, A_>)$

Worst case. If a is always the smallest or largest element, the list shrinks by only 1 per step. The recursion depth is $n - 1$ and each level costs $\Theta(n)$, so the runtime is $\Theta(n^2)$.

2.12 Randomized Pivot

Definition (good partition). The pivot a gives a **good partition** if $n/4 \le \text{rank}(a) \le 3n/4$. If the rank is not unique, a is good if one of its possible ranks lies in this range.



- A uniformly random pivot is good with probability $\ge 1/2$, since half of all ranks lie in the middle range.
- After a good partition, $A_<$ and $A_>$ have at most $3n/4$ elements: the list shrinks to at most $\frac{3}{4}$ of its size.

Theoretical algorithm for the analysis. Each round has two phases:

1. Phase A: repeatedly sample a pivot until finding a good one.
2. Phase B: make the partition and proceed to $A_<$ or $A_>$, or terminate.

Phase A is a Bernoulli experiment with success probability $p \ge 1/2$, so it needs $1/p \le 2$ trials in expectation. Each trial costs $O(n)$, hence $E(\text{runtime of phase A}) = O(n)$.

Expected runtime. Let $T(n)$ be the maximal expected time over lists of size at most n, so T is monotone. Then

$$T(n) \le T\left(\left\lceil \tfrac{3n}{4} \right\rceil\right) + O(n).$$

Master theorem with $a = 1$, $b = 4/3$, $f(n) = \Theta(n)$: $\gamma = \log_{4/3} 1 = 0$ and the exponent of f is $1 > \gamma$, so Case III gives $\boldsymbol{T(n) = \Theta(n)}$. Unrolling gives the same: $cn \cdot (1 + \frac{3}{4} + \frac{9}{16} + \dots) = 4cn$.

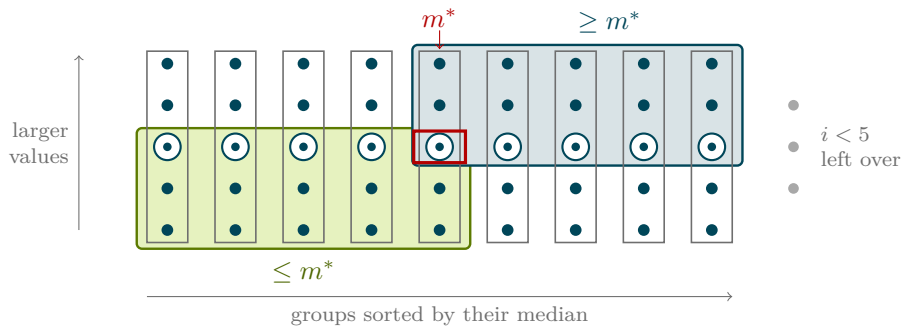
Remark (in practice). In practice we use *every* pivot, not only good ones: we simply partition and recurse. The analysis goes through as well and still gives expected time $O(n)$.

2.13 Deterministic Algorithm: Median of Medians

Blum, Floyd, Pratt, Rivest and Tarjan (1973) compute a good pivot deterministically:

1. Partition A into groups of 5 and determine the median $m_1, m_2, \dots, m_{\lfloor n/5 \rfloor}$ of each group.
2. Determine recursively the median m^* of the medians.
3. Use m^* as the pivot.

Step 1 costs $O(n)$, since every group has constant size. Step 2 is a recursive call on $\lfloor n/5 \rfloor \approx 0.2\,n$ medians.



Each column is a group of 5, larger values on top, its median circled; the columns are ordered by their medians, so m^* (red) is the median of the middle column. Everything in the green box is $\leq m^*$, everything in the blue box $\geq m^*$. The sorting is only for the picture; the algorithm sorts nothing.

How much do we get rid of? At least half of the $\lfloor n/5 \rfloor$ medians are $\leq m^*$, and each brings 3 elements $\leq m^*$ (itself and the two below it). The same holds for $\geq m^*$. So we get rid of $\geq 3 \cdot \lfloor n/5 \rfloor \cdot \frac{1}{2} \approx 0.3\,n$ elements, and the size after step 3 is at most about $0.7\,n$.

Exact bound. Assume we go to $A_>$. Write $n = 5\ell + i$ with $i \in \{0, \ldots, 4\}$. At least $3\ell/2$ elements are $\leq m^*$, and none of them is in $A_>$:

$$n_> \leq n - 3 \cdot \frac{\ell}{2} = n - 3 \cdot \frac{n-i}{10} = \frac{7n}{10} + \underbrace{\frac{3i}{10}}_{<2}.$$

Hence $n_> \leq 7n/10 + 2$, and by symmetry the same holds for $n_<$.

Linear runtime. Let $T(n)$ be the worst-case runtime on lists of size at most n (so T is monotone). With $\lfloor 7n/10 \rfloor + 3$ as a safe integer bound for the second call:

$$T(n) \leq T\left(\left\lfloor \tfrac{n}{5} \right\rfloor\right) + T\left(\left\lfloor \tfrac{7n}{10} \right\rfloor + 3\right) + cn.$$

The master theorem does not apply (two calls of different size), so we use induction.

Theorem 2.3 (Linear-time selection). There is a constant D with $T(n) \leq Dn$ for all $n \geq 1$.

Proof. Choose $D \geq 20c$, and D big enough that $T(n) \leq Dn$ for $1 \leq n \leq 60$. For $n > 60$ both arguments are smaller than n, so by induction:

$$\begin{aligned} T(n) &\leq D\left\lfloor \tfrac{n}{5} \right\rfloor + D\left(\left\lfloor \tfrac{7}{10}n \right\rfloor + 3\right) + cn \\ &\leq D\,\tfrac{n}{5} + D\,\tfrac{7}{10}n + \tfrac{n}{20}D + cn \qquad (3 \leq \tfrac{n}{20} \text{ for } n \geq 60) \\ &= Dn\,(0.2 + 0.7 + 0.05) + cn \\ &\leq Dn\,(0.95 + 0.05) = Dn \qquad (cn \leq 0.05\,Dn \text{ since } D \geq 20c). \end{aligned}$$

■

The key is $\frac{1}{5} + \frac{7}{10} < 1$: both subproblems together are smaller than the original, so the work per level shrinks geometrically.

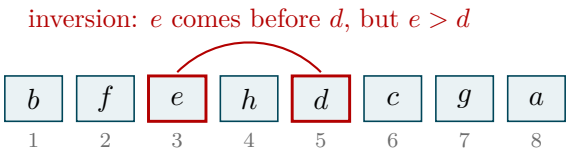
Lecture of 30.10.2026 ▪ Inversions, extended Mergesort, weighted inversions, search trees

The last application of divide and conquer measures how far a list is from being sorted: we count its **inversions**, the pairs of elements in the wrong order. Checking all pairs costs $O(n^2)$. Mergesort (Section 2.1) can count them on the fly in $O(n \log n)$: the merge step sees exactly the pairs between the two halves, and the master theorem gives the runtime (Case II, as for Mergesort itself). A second $O(n \log n)$ method uses a search tree with rank queries (rank as in Section 2.10). On the way we count a weighted variant and see why inversions measure the work of insertion sort.

D **Counting Inversions**

Lecture 30.10.2026 ▪ extended Mergesort, weighted inversions, insertion sort, search trees

2.14 Inversions



The letters are ordered alphabetically, $a < b < \cdots < h$. In the list (b, f, e, h, d, c, g, a) the pair e, d is *out of order*: e stands before d, although it is larger.

Definition 2.4 (Inversion). Let $(a_1, a_2, \ldots, a_n)$ be a list. An **inversion** is a pair of indices (i, j) with

$$1 \le i < j \le n \qquad \text{and} \qquad a_i > a_j,$$

i.e. a pair of items that are out of order. I denotes the number of inversions.

Assumption: all elements are distinct. Then every pair $i < j$ is either in order ($a_i < a_j$) or inverted ($a_i > a_j$).

The number of inversions is a **measure of sortedness**:

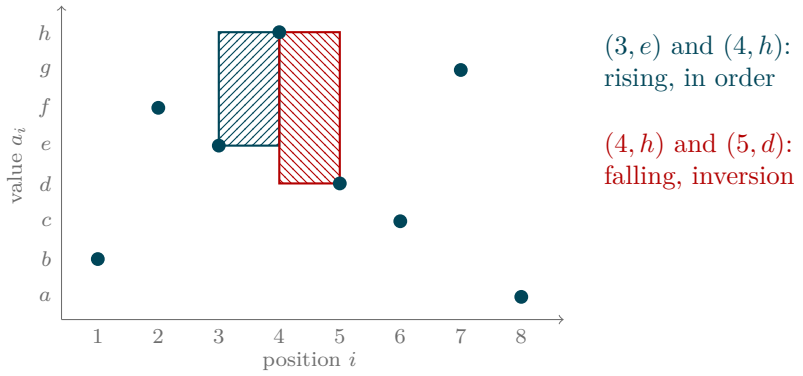
- a sorted list has $I = 0$;
- a reverse sorted list has $I = \binom{n}{2} = \frac{n(n-1)}{2}$, since *every* pair is out of order – the maximum.

The example list has $I = 16$ of $\binom{8}{2} = 28$ pairs.

Problem: count the inversions of a given list. The trivial algorithm checks all $\binom{n}{2}$ pairs: $O(n^2)$. **Goal:** $O(n \log n)$.

2.14.1 A Geometric View

Draw every element as a point (i, a_i). Two points span a rectangle: the pair is in order if the rectangle *rises* from left to right, and an inversion if it *falls*.



The trivial algorithm looks at the rectangle of every pair. Flipping an inverted pair (swapping the two values) removes this inversion; sorting removes them until no falling rectangle is left.

2.15 Counting Inversions with Mergesort

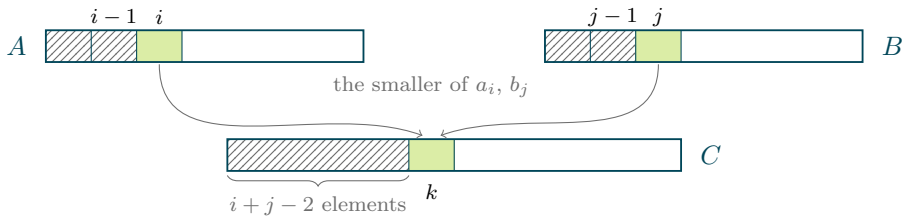
Split the list into a left part A and a right part B. Every inversion lies inside A, inside B, or *between* them (a_i in A, b_j in B). The first two kinds are counted recursively. The new task is to count the pairs between

$$\underbrace{(a_1, a_2, \ldots, a_m)}_{\text{left}}\ \underbrace{(b_1, b_2, \ldots, b_n)}_{\text{right}} \qquad \text{with } a_i > b_j$$

(m, n = lengths of the two lists). Every a_i stands before every b_j, so the order *inside* A and B does not matter for these pairs. We may sort both lists recursively first and count while merging.

2.15.1 Counting While Merging

During the merge, i points to the next element of A and j to the next element of B. The merged list C already holds $i-1$ elements of A and $j-1$ of B, so the next position is $k = i + j - 1$:



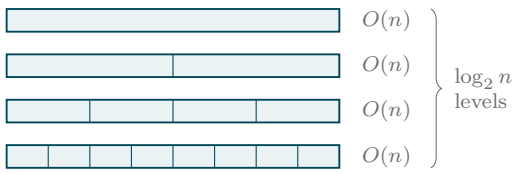
Key observation. When a_i is taken, $b_1, \ldots, b_{j-1}$ are already in C: they are smaller than a_i but stand after it in the original list. So a_i forms exactly $j-1$ inversions with B (the waiting $b_j, b_{j+1}, \ldots$ are larger). Every inversion between the lists is counted once, when its element from A is taken.

merge(A, B) *(A, B sorted)*
  $i := 1,\ j := 1,\ I := 0,\ S := 0,\ \bar{I} := 0$
  for $k = 1, \dots, m+n$:
    if $j = n+1$ **or** $(i \le m$ **and** $a_i < b_j)$: *(take from A)*
      $c_k := a_i, \quad I := I + (j-1)$
      $\bar{I} := \bar{I} + S - (j-1) \cdot p_i$
      $i := i + 1$
    else: *(take from B)*
      $c_k := b_j, \quad S := S + q_j$
      $j := j + 1$
  return $(c_1, \dots, c_{m+n}),\ I$ (and $\bar{I}$)

The gray lines are for the weighted variant (Section 2.16). $j = n+1$ means B is exhausted: the rest of A is copied, each element with n inversions. The merge costs $O(m+n)$; counting adds $O(1)$ per step.

2.15.2 Embedding into Mergesort

MS(L):
  if $|L| \le 1$: **return** $(L, 0)$
  split $L = A + B$ evenly
  $(A', I_A) := \text{MS}(A)$
  $(B', I_B) := \text{MS}(B)$
  $(C, I_{AB}) := \text{merge}(A', B')$
  return $(C,\ I_A + I_B + I_{AB})$



The recursion is the one of Mergesort, so with the master theorem (Case II, $a = b = 2$, $f(n) = \Theta(n)$):

$$T(n) = 2\,T(\tfrac{n}{2}) + O(n) \quad \Rightarrow \quad T(n) = \Theta(n \log n).$$

Example (the list (b, f, e, h, d, c, g, a)). Halves (b, f, e, h) with $I_A = 1$ (pair f, e) and (d, c, g, a) with $I_B = 4$ (pairs d, c; d, a; c, a; g, a). Sorted: $A' = (b, e, f, h)$, $B' = (a, c, d, g)$. When b is taken, only a has left B' $(+1)$; when e and f are taken, a, c, d have $(+3, +3)$; when h is taken, all four $(+4)$. So $I_{AB} = 11$ and $I = 1 + 4 + 11 = 16$.

Theorem 2.5 (Counting inversions). The number of inversions of a list of n distinct elements, and its total weight (Section 2.16), can be computed in $O(n \log n)$ time.

2.16 Variation: Weighted Inversions

An inversion (i, j) now gets the weight $j - i \geq 1$: pairs far apart count more. We want

$$\bar{I} = \sum_{\substack{i<j \\ a_i > a_j}} (j - i), \qquad 0 \leq \bar{I} \leq \sum_{i=1}^{n} \sum_{j=i+1}^{n} (j - i) = \sum_{d=1}^{n-1} d\,(n - d) = \frac{n^3 - n}{6} \leq n^3$$

(there are $n - d$ pairs at distance d; the maximum is the reverse sorted list). So $\bar{I}$ is a polynomial $an^3 + bn^2 + \ldots$ with $a = \frac{1}{6}$ and fits into a constant number of words.

Keep track of the original position. Mergesort moves the elements, but the weight depends on where they stood. Initially attach $R := (1, 2, \ldots, n)$: element a_i carries $r = i$. In the merge, p_i is the original position of a_i and q_j that of b_j.

When a_i is taken, it forms inversions with $b_1, \ldots, b_{j-1}$, with weights $q_l - p_i > 0$. With $S = q_1 + \cdots + q_{j-1}$, the sum of the original positions of the B-elements already taken,

$$\sum_{l=1}^{j-1} (q_l - p_i) = S - (j - 1) \cdot p_i.$$

These are the gray lines of merge. Mergesort adds $\bar{I}_A + \bar{I}_B + \bar{I}_{AB}$; the runtime stays $O(n \log n)$.

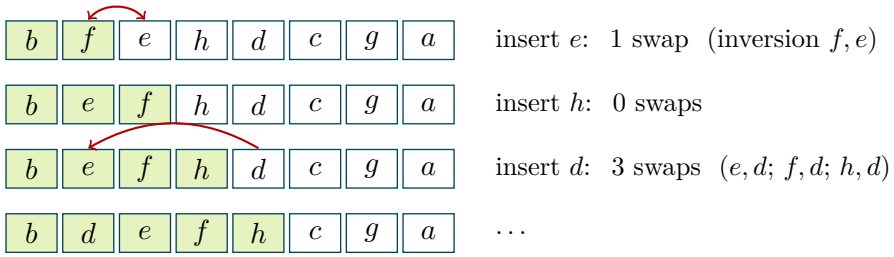
Example (top-level merge of the list). $A' = (b, e, f, h)$ with $p = (1, 3, 2, 4)$, $B' = (a, c, d, g)$ with $q = (8, 6, 5, 7)$:

k	taken	from	I	S	$\bar{I}$
1	a	B	0	8	0
2	b	A	1	8	$0 + 8 - 1 \cdot 1 = 7$
3	c	B	1	14	7
4	d	B	1	19	7
5	e	A	4	19	$7 + 19 - 3 \cdot 3 = 17$
6	f	A	7	19	$17 + 19 - 3 \cdot 2 = 30$
7	g	B	7	26	30
8	h	A	11	26	$30 + 26 - 4 \cdot 4 = 40$

With $\bar{I}_A = 1$ and $\bar{I}_B = 7$ from the halves, the total weight is $1 + 7 + 40 = 48$.

2.17 Inversions and Insertion Sort

Insertion sort takes the elements from left to right and swaps each one with its left neighbour until it is in place (green: already sorted):



A swap of two neighbours that are out of order removes exactly one inversion and creates none, and insertion sort swaps only such neighbours. Every insertion ends with at most one comparison without a swap. Hence

$$I = \text{number of swaps}, \qquad I \le \text{number of comparisons} \le I + n - 1,$$

i.e. $I \approx$ number of comparisons $- n$. Example: 16 swaps, 22 comparisons. So insertion sort runs in $O(n + I)$: fast on almost sorted lists, but $\Theta(n^2)$ in the worst case – no help for counting.

2.18 Counting with a Search Tree

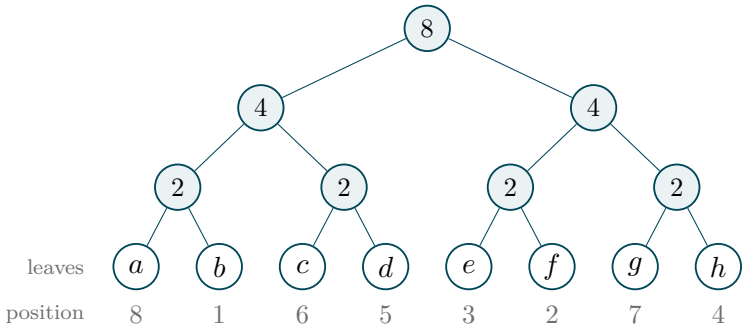
Go through the list from left to right and keep the elements seen so far in a search tree T:

$I := 0,\ T := \emptyset$
for $i = 1$ **to** n:
  insert a_i into T
  $I := I + (i - \text{rank}(a_i \text{ in } T))$

After the insertion $T = \{a_1, \ldots, a_i\}$. Of these, $\text{rank}(a_i) - 1$ are smaller than a_i, so $i - \text{rank}(a_i)$ are larger and stand before it: each is an inversion (l, i), $l < i$. Every inversion is counted once, at its right element. A search tree with insertion and **rank queries** in $O(\log n)$ gives $O(n \log n)$ in total.

2.18.1 Example

The tree keeps the elements in its leaves, in sorted order; every node stores the number of leaves below it. Under each leaf: the position of the element in the list.



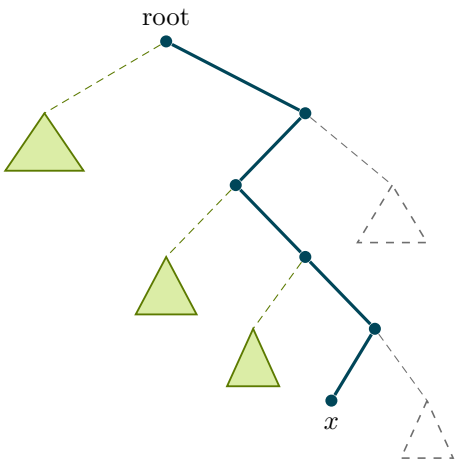
In the lecture the list was processed from the back: in step i the tree holds exactly $a_1, \ldots, a_i$, so we can start with the full tree and remove a_i after step i (Section 2.18.3). The added terms are the same:

i	a_i	T	$\text{rank}(a_i \text{ in } T)$	add $i - \text{rank}$
8	a	$\{a, b, \ldots, h\}$	1	7
7	g	$\{b, c, \ldots, h\}$	6	1
6	c	$\{b, c, d, e, f, h\}$	2	4
5	d	$\{b, d, e, f, h\}$	2	3
4	h	$\{b, e, f, h\}$	4	0
3	e	$\{b, e, f\}$	2	1
2	f	$\{b, f\}$	2	0
1	b	$\{b\}$	1	0
				$I = 16$

2.18.2 Rank Queries

With the leaf counts, the rank of x is read off the path from its leaf to the root:

$$\begin{aligned}\operatorname{rank}(x) &= 1 + \text{number of leaves left of } x \\ &= 1 + \text{number of leaves in subtrees branching off to the left of this path.}\end{aligned}$$



green: subtrees left of the path, their leaves are counted

dashed: subtrees right of the path, not counted

Where the path enters a node from its *right* child, all leaves of the left child are smaller than x; where it enters from the left child, the right subtree holds only larger elements. So we add up the stored counts of the left siblings on the way up. In a balanced tree the path has length $O(\log n)$.

2.18.3 Backwards: Removing Instead of Inserting

All elements are known in advance, so insertions (and their rebalancing) are not needed: sort the list, build a balanced tree over the sorted elements once, and maintain in every node the number of leaves below it.

$I := 0$
for $i = n, n-1, \ldots, 1$:
  determine the rank of a_i
  $I := I + (i - \text{rank})$
  remove a_i and update the node counts on its path to the root

Removing a_i decreases the counts on its path by 1; the shape of the tree never changes. Each step costs $O(\log n)$, sorting $O(n \log n)$, so in total $O(n \log n) + O(n \log n) = O(n \log n)$.

E **Overview**

All algorithms of the chapter at a glance

2.19 Summary of Divide and Conquer

Every algorithm of this chapter splits the input, solves the parts recursively and combines the results. Its recurrence is solved by a recursion tree, expansion, induction or the master theorem (Theorem 2.2).

Algorithm	Recurrence	Solved by	Result	Section
Mergesort	$2T(n/2) + \Theta(n)$	Case II	$\Theta(n \log n)$	2.1, 2.2
D&C multiplication	$4T(n/2) + \Theta(n)$	Case I	$\Theta(n^2)$	2.3
Karatsuba	$3T(n/2) + \Theta(n)$	Case I, induction	$\Theta(n^{\log_2 3})$	2.3
Quickselect, good pivot	$T(\lceil 3n/4 \rceil) + \Theta(n)$	Case III	$\Theta(n)$ expected	2.12
Median of medians	$T(\lfloor n/5 \rfloor) + T(\lfloor 7n/10 \rfloor + 3) + cn$	induction	$\Theta(n)$	2.13
Counting inversions	$2T(n/2) + \Theta(n)$	Case II	$\Theta(n \log n)$	2.15

Two lessons recur. First, the gain comes from the combine step or from fewer subproblems: Karatsuba saves one of four multiplications, the median-of-medians pivot guarantees that both calls together are smaller than n ($\frac{1}{5} + \frac{7}{10} < 1$). Second, when the master theorem does not fit (two calls of different size, extra constants), induction with a suitably strengthened claim or a shift of domain or value (Sections 2.8, 2.9) does the job.

Notes on Corrections

These deviate from the handwritten notes or the official slides and should be checked against the lecture.

From the notes of 19.10.2026:

- Induction base Mergesort: $A \geq C_0/2$ (not $C_0/4$).
- Karatsuba: $b := x_H \cdot y_H$ (notes say $x_H \cdot x_H$).
- Strengthened claim: $T(n) \leq A \cdot n^\gamma - Dn$ with minus (notes and slides say $+\,Dn$).
- $\log_2 3 \approx 1.585$ (slides: ≈ 1.7).
- Master theorem: $b > 1$ and $\gamma \geq 0$ (slides: $\gamma > 0$); Case 2 with Θ. Case 3 holds for $f = \Theta(n^{\gamma+\varepsilon})$ as on the slides; with only $f = \Omega(n^{\gamma+\varepsilon})$ the regularity condition is needed in addition. (Corrected on 30.10.: earlier versions of this script wrote Ω without it.)

From the notes of 22.10.2026:

- $\gamma = \log_b a \geq 0$ (notes: > 0); $a = 1$ gives $\gamma = 0$.
- Case III: the notes write $f = \Theta(n^{\gamma+\delta})$ or the regularity condition. Both are correct; the regularity condition alone even implies $f = \Omega(n^{\gamma+\delta})$. Only with the weaker $f = \Omega(n^{\gamma+\delta})$ is the regularity condition needed in addition (Remark after Theorem 2.2).
- Regularity example: with $\lceil n/2 \rceil \leq (n+1)/2$ the bound $1/2$ holds from $n \geq 5$; the notes' estimate $2^{-n/2+2}$ reaches $1/2$ only from $n \geq 6$.
- Gap example: the solution is $\Theta(n \log^2 n)$ (notes: $n \log_2 n$), as the extended Case II confirms.
- Shifting the domain: $v = -2$, so $B(n) = A(n+2)$ and $A(n) = B(n-2)$ (notes: $v = 2$, $A(n) = B(n+2)$). With $v = 2$ the argument becomes $n/2 + 2$, not $n/2$.
- Step 2: "enforcing a bigger n_0" (notes: "a bigger a").

From the notes of 26.10.2026:

- Goal: "Selecting in $O(n)$" (notes: "Sorting in $O(n)$"); sorting by comparisons needs $\Omega(n \log n)$.
- Quickselect, second case: return a if $k \leq n_< + n_=$ (notes: $k \leq n_<$). Third case: recurse into $A_>$ otherwise, i.e. $k > n_< + n_=$ (notes: $k \leq n_< + n_=$).
- Good partition: the size decreases *to* at most $\frac{3}{4}$ of $|A|$, i.e. *by* at least $\frac{1}{4}$.
- Phase A terminates after ≤ 2 iterations in expectation (notes: "often").
- Master theorem for Quickselect: $b = 4/3$ and $\gamma = 0$; the exponent of f is $1 > \gamma$, hence Case III.
- Exact bound: $n - 3 \cdot \ell/2 = n - 3 \cdot (n-i)/10$ (notes: $(n-1)/10$), and the conclusion is about $n_>$ (we assumed $A_>$); $n_<$ follows by symmetry.
- $\lfloor 7n/10 \rfloor + 2$ already suffices as recursion size; the $+3$ in the notes is a safe overestimate.
- Added: T is the maximum over all lists of size at most n, so T is monotone; both recurrences need this.
- Final step: $Dn\,(0.95 + 0.05) = Dn$ (notes: $0.95 + 0.5$). The conditions are $n \geq 60$ for $3D \leq \frac{n}{20} D$ and $D \geq 20c$ for $cn \leq \frac{n}{20} D$.

From the notes of 30.10.2026:

- Indices: the list is $(a_1, \ldots, a_n)$ and an inversion needs $i < j$ strictly. With $(a_0, \ldots, a_n)$ as in the notes there are $n+1$ elements, and a reverse sorted list would have $\binom{n+1}{2}$ inversions instead of $\binom{n}{2}$.
- Assumption: all elements are *distinct* (notes: "distant").
- The hatched rectangle in the notes belongs to $(3, e)$ and $(4, h)$; it rises, so this pair is in order. The inverted pair $(4, h)$, $(5, d)$ was added for comparison.
- The two lists are $(a_1, \ldots, a_m)$ and $(b_1, \ldots, b_n)$ (notes: both from index 0 to n); the loop runs over $k = 1, \ldots, m+n$ and returns $(c_1, \ldots, c_{m+n})$.
- Merge condition: take from A if B is exhausted ($j = n+1$) or $a_i < b_j$, and only while A is not exhausted ($i \leq m$); the case $i = m+1$ is missing in the notes.
- Weighted update: $\bar{I} := \bar{I} + S - (j-1) \cdot p_i$, computed before $i := i+1$; each of the $j-1$ pairs has weight $q_l - p_i > 0$. In the else branch, S adds the original *position* q_j of b_j, not its value.
- Mergesort returns $I_A + I_B + I_{AB}$ (notes: I_C), and the base case returns $(L, 0)$.
- The double sum for the maximal weight is over $(j-i)$ (notes: $(j-1)$); its exact value is $\frac{n^3-n}{6}$.
- "Inversionsort" is insertion sort. The number of comparisons lies between I and $I+n-1$.
- "rank $= 1 +$ number of leaves *left of* the element", and the update in the backwards loop is $I := I + (i - \text{rank})$, as in the forward loop.
- All algorithms of this lecture were checked against counting all pairs on 3000 random lists; the example list has $I = 16$ and total weight $\bar{I} = 48$.

Review of the whole chapter (30.10.2026):

- Theorem 2.2, Case 3: $f = \Theta(n^{\gamma+\varepsilon})$ or the regularity condition (it said Ω alone, which is not enough; see the corrected note of 19.10.).
- Step 3: the lower bound $T^{low} \leq T$ and the monotonicity of T^{low} were added.
- Step 5: the figure for the shift by v was added (it was only in the notes).
- Karatsuba (2.3.3): the geometric sum behind $T(n) = O(3^k)$ is now written out.

Chapter 3

Dynamic Programming

Lecture of 02.11.2026 ▪ Knapsack problem, DP table, memoization, saving space

Today we start the second design principle. Divide and conquer splits a problem into *independent* subproblems. In dynamic programming the subproblems overlap, so we solve each of them only once and store its solution in a table. The running example is the knapsack problem: we set up subproblems, a recursion and a table, read the optimal solution off the table, and see why the runtime $O(nB)$ is only pseudopolynomial. Then we look at the same algorithm top-down (memoization) and at a divide-and-conquer trick that needs only $O(B)$ space.

A The DP Method on Knapsack

Lecture 02.11.2026 ▪ subproblems, recursion, table, backtracking, analysis

3.1 The Knapsack Problem

Definition 3.1 (Knapsack problem, Rucksackproblem). Given n items, item i with weight a_i and value v_i, and a capacity B. Find a subset $S \subseteq \{1, \dots, n\}$ with

$$\sum_{i \in S} a_i \le B \qquad \text{and} \qquad \sum_{i \in S} v_i \to \max .$$

Assumption: the weights are integers, $a_i \in \mathbb{N}$ (also $B \in \mathbb{N}$); the values may be real, $v_i \in \mathbb{R}$. The integer weights are what we use as table indices below.

Example (the example of the lecture, $B = 12$).

i	1	2	3	4	5
a_i	4	3	5	2	6
v_i	7	8	6	3	9

Trying all 2^n subsets is hopeless for large n. Instead we solve the problem for fewer items and smaller capacities first.

3.2 The Dynamic Programming Recipe

1) Define subproblems.

2) Formulate a recursion that solves a subproblem with the help of smaller subproblems.

3) Solve the subproblems in a systematic order: initialization, plus two nested loops (columnwise or rowwise).

Then **recover the solution**, not only its value (Section 3.2.4). The lecture stressed that you must be able to *apply* this in the exam: set up the table and fill it by hand.

3.2.1 Step 1: Subproblems

> **Definition 3.2 (Subproblem $P_i(b)$).** $P_i(b)$ = the largest value that can be obtained from the items $1, \dots, i$ with total weight at most b, for $i = 0, \dots, n$ and $b = 0, \dots, B$.

The original problem is $P_n(B)$. There are $(n+1)(B+1)$ subproblems.

3.2.2 Step 2: The Recursion

For item i there are two choices. Either we do not take it; then the best value is $P_{i-1}(b)$. Or we take it, which is only possible if $a_i \le b$; then the other items may weigh at most $b - a_i$, and the best value is $P_{i-1}(b - a_i) + v_i$:

$$P_i(b) = \begin{cases} \max\{ \underbrace{P_{i-1}(b)}_{\text{skip } i}, \underbrace{P_{i-1}(b-a_i) + v_i}_{\text{take } i} \} & \text{if } b \ge a_i, \\ P_{i-1}(b) & \text{if } b < a_i, \end{cases} \qquad \text{initial condition: } P_0(b) = 0 \text{ for all } b.$$

3.2.3 Step 3: The DP Table

Column i only needs column $i-1$, so we fill the table column by column, each column from $b = 0$ to B:

for $b = 0, \dots, B$: $P_0(b) := 0$
for $i = 1, \dots, n$:
  for $b = 0, \dots, B$:
    $P_i(b) := P_{i-1}(b)$
    if $b \ge a_i$ **and** $P_{i-1}(b - a_i) + v_i > P_i(b)$: $P_i(b) := P_{i-1}(b - a_i) + v_i$
return $P_n(B)$

$b\backslash i$	**0**	**1**	**2**	**3**	**4**	**5**
0	0	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	0	0	3	3
3	0	0	8	8	8	8
4	0	7	8	8	8	8
5	0	7	+8	8	11	11
6	0	7	8	8	11	11
7	0	7	15	15	15	15
8	0	7	15	15	15	15
9	0	7	15	15	18	18
10	0	7	15	15	18	18
11	0	7	15	15	18	20
12	0	7	15	21	21	21

$P_2(7) = \max\{7,\ 7+8\}$

backtracking:
take 3, 2, 1

How the table is built. Every entry looks at two entries of the previous column: the one in the same row (skip i) and the one a_i rows higher (take i, plus v_i). Some entries worked out:

- $P_1(b) = 7$ for $b \geq 4 = a_1$, and 0 below: one item fits or not.
- $P_2(7) = \max\{P_1(7),\ P_1(7-3)+8\} = \max\{7,\ 7+8\} = 15$ (blue arrows): items 1 and 2 together weigh exactly 7.
- $P_4(5) = \max\{P_3(5),\ P_3(3)+3\} = \max\{8,\ 8+3\} = 11$.
- $P_3(12) = \max\{P_2(12),\ P_2(7)+6\} = \max\{15,\ 15+6\} = 21$.
- $P_5(11) = \max\{P_4(11),\ P_4(5)+9\} = \max\{18,\ 11+9\} = 20$.

The optimal value is $P_5(12) = 21$.

3.2.4 Recovering the Solution

The table only contains values. To get the set S there are two ways:

a) For each entry, remember the way how it was constructed (skip or take).

b) Recompute this information by reevaluating the recursion (*backtracking*): start at $P_n(B)$ and go back column by column.

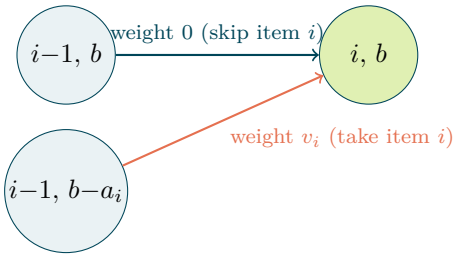
Backtracking in the example (orange): $P_5(12) = 21 = P_4(12)$, so item 5 is skipped; $P_4(12) = P_3(12)$, item 4 skipped; $P_3(12) = 21 \neq P_2(12) = 15$, so item 3 is taken and we continue at $b = 12 - 5 = 7$; $P_2(7) = 15 \neq P_1(7) = 7$, item 2 taken, $b = 4$; $P_1(4) = 7 \neq P_0(4) = 0$, item 1 taken.

$$S = \{1, 2, 3\}, \qquad a_1 + a_2 + a_3 = 4 + 3 + 5 = 12 \leq B, \qquad v_1 + v_2 + v_3 = 7 + 8 + 6 = 21.$$

3.2.5 DP as a Path Problem

Often, DP algorithms can be interpreted as finding a longest or shortest path in an acyclic graph:

- nodes = subproblems, here the table entries (i, b);
- arcs = the relations expressed in the recursion (step 2).



$P_i(b)$ is the length of a longest path that ends in (i, b), and the graph is acyclic because every arc goes from layer $i-1$ to layer i. Filling the table column by column processes the nodes in a topological order.

3.2.6 Analysis

Theorem 3.3 (Knapsack by DP). The knapsack problem with integer weights can be solved in $O(nB)$ time and $O(nB)$ space.

Each of the $(n+1)(B+1)$ entries costs $O(1)$; backtracking adds $O(n)$.

Remark (pseudopolynomial time). $O(nB)$ looks efficient for small B, but for large B it can be quite time consuming. The runtime is polynomial in the *size of the numbers* (B), not in their length $\log_2 B$, which is what the input size counts: doubling the number of bits of B squares B. Such a runtime is called **pseudopolynomial**. Knapsack is NP-hard (later in the course), so we cannot expect a truly polynomial algorithm.

B Variations and Implementation

Lecture 02.11.2026 ▪ swapped roles, memoization, saving space

3.3 Variation: Minimum Weight for a Given Value

A change of viewpoint (*quasi Umkehrung*, another angle) swaps the roles of weights and values. Now the values are integers, $v_i \in \mathbb{N}$, and the weights real, $a_i \in \mathbb{R}_{\geq 0}$.

Definition 3.4 (Subproblem $Q_i(c)$). $Q_i(c)$ = the smallest total weight of a subset of the items $1, \ldots, i$ with which a total value of exactly c can be reached (∞ if no subset has value c).

$$Q_i(c) = \begin{cases} \min\{\, Q_{i-1}(c),\ Q_{i-1}(c - v_i) + a_i \,\} & \text{if } c \geq v_i, \\ Q_{i-1}(c) & \text{if } c < v_i, \end{cases} \qquad Q_0(0) = 0, \quad Q_0(c) = \infty \text{ for } c > 0.$$

The original problem is answered by

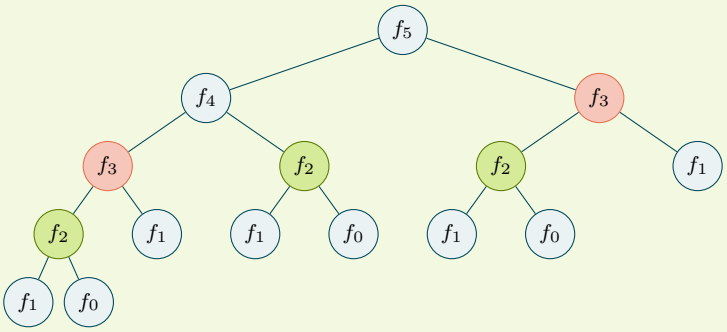
$$\max\{\, c \mid Q_n(c) \le B \,\},$$

the largest value whose cheapest subset still fits. With $V = \sum_i v_i$ the table has $(n+1)(V+1)$ entries: $O(nV)$ time. In the example, $\max\{c \mid Q_5(c) \le 12\} = 21$ again.

Remark (another variation). "Value of *at least* c" instead of "exactly c": replace $Q_{i-1}(c-v_i)$ by $Q_{i-1}(\max\{c-v_i, 0\})$; then Q_i is monotone in c.

3.4 Top-Down: Memoization

Example (Fibonacci numbers). $f(n) = f(n-1)+f(n-2)$. Computed directly by recursion, the same values are computed again and again:



$f(3)$ is computed twice, $f(2)$ three times. The number of calls grows like 1.618^n, although there are only $n+1$ different subproblems.

Memoization: solve the recursive subproblems as needed, top-down, but remember the solutions once they are solved (in a hash table). For the knapsack recursion

$$\underbrace{P_i(b)}_{P(i,b)} = \max\{\, P_{i-1}(b),\ \underbrace{P_{i-1}(b-a_i)+v_i}_{\text{table lookup}} \,\},$$

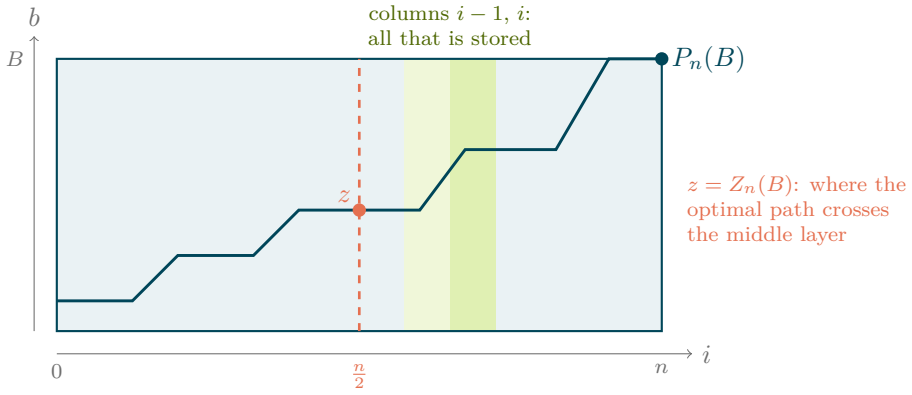
the key of the hash table is the pair (i, b). For every subproblem:

- check whether the result is in the table;
- if yes, report it;
- if not, compute it recursively and store it in the table.

So every subproblem is solved at most once. Memoization only touches the subproblems that are actually needed for $P_n(B)$; bottom-up fills the whole table but avoids recursion and hashing.

3.5 Saving Space

To compute the *value* $P_n(B)$ we only need the previous column: two columns of length $B+1$, so $O(2B) = O(B)$ space. But then the information for backtracking is gone. In the path view: we look for a longest path from s to t in a layered graph where all arcs go from layer i to layer $i+1$.



Divide and conquer: remember through which node z on the middle layer $\frac{n}{2}$ the path went. For $i > \frac{n}{2}$, store along with $P_i(b)$ the value $Z_i(b)$, starting with $Z_{n/2}(b) = b$:

$$\begin{array}{lll} \textbf{if } b < a_i \textbf{ or } P_{i-1}(b) \geq P_{i-1}(b-a_i) + v_i: & P_i(b) = P_{i-1}(b), & Z_i(b) = Z_{i-1}(b), \\ \textbf{else}: & P_i(b) = P_{i-1}(b-a_i) + v_i, & Z_i(b) = Z_{i-1}(b-a_i). \end{array}$$

Then look at $z = Z_n(B)$: the optimum path went through the node $Z_n(B)$ on the middle level. So an optimal solution consists of an optimal solution for items $1, \ldots, \frac{n}{2}$ with capacity z and items $\frac{n}{2}+1, \ldots, n$ with total weight $B - z$. Both halves are solved recursively in the same way, and their chosen items together form S.

- **Space:** $O(B)$ (two columns of P and of Z).
- **Runtime:** $O(nB \log n)$: every level of the recursion costs $O(nB)$, and there are $\log_2 n$ levels.

> **Remark (sharper bound).** On recursion level j there are 2^j subproblems with $\frac{n}{2^j}$ items each, and their capacities add up to at most B. So level j costs $O(\frac{n}{2^j}(B + 2^j))$, and the sum over all levels is $O(nB + n \log n)$ – the same as the plain table, up to the $n \log n$ term. (This is the idea of Hirschberg's algorithm.)

Lecture of 06.11.2026 ▪ Optimal binary search trees, polygon triangulation, traveling salesman

Today we apply the dynamic programming recipe of the last lecture to three more problems. All three have the same pattern: a subproblem is described by an *interval* or a *set*, and the recursion tries every possible "split point" and keeps the best one. For optimal binary search trees and polygon triangulation the subproblems are intervals (i, j), which gives $O(n^2)$ subproblems and $O(n^3)$ time. For the traveling salesman problem the subproblems are sets of vertices, which gives an exponential, but still much better than $(n-1)!$, algorithm.

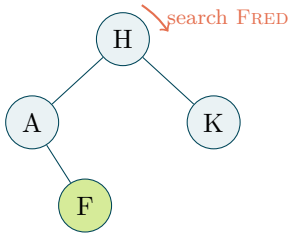
C **Interval DP: Search Trees and Triangulations**

Lecture 06.11.2026 ▪ optimal binary search trees, minimum weight triangulation

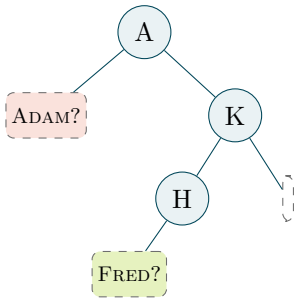
3.6 Optimal Binary Search Trees

3.6.1 The Problem

Suppose we store keys such as Alice, Fred, Kate, Harold, … in a binary search tree. To find Fred we start at the root and compare: at every node we go left or right until we find the key. A search for a key that is *not* stored, such as Adam, ends in an empty leaf, a *gap* between two stored keys.



Fred found after 3 comparisons



unsuccessful searches end in a gap

If some keys are searched much more often than others, they should be close to the root. For example, Adam is a fairly common name, so the gap where its search ends should not be deep in the tree.

Definition 3.5 (Optimal binary search tree). Given n keys $x_1 < x_2 < \cdots < x_n$ with *frequencies*

- p_i = frequency of searching for x_i $\quad (i = 1, \ldots, n)$,
- q_i = frequency of searching for some $x \in (x_i, x_{i+1})$ $\quad (i = 0, \ldots, n)$,

where $x_0 = -\infty$ and $x_{n+1} = +\infty$. Find the binary search tree that minimizes the total number of comparisons when searching according to the given frequencies:

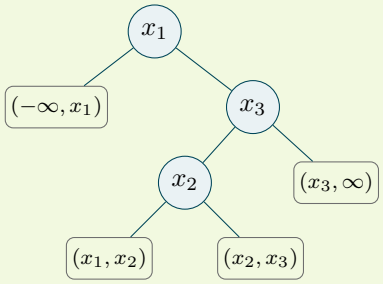
$$\sum_{i=1}^{n} p_i(\operatorname{depth}(x_i) + 1) \;+\; \sum_{i=0}^{n} q_i \operatorname{depth}((x_i, x_{i+1})).$$

	x_1		x_2			x_n	
q_0	p_1	q_1	p_2	q_2	$\cdots$	p_n	q_n
$(-\infty, x_1)$		(x_1, x_2)		(x_2, x_3)			(x_n, ∞)

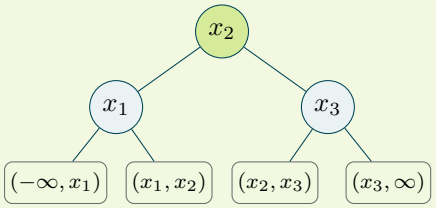
Why these depths? The root has depth 0. A successful search for x_i compares with every node on the path to x_i and with x_i itself: $\operatorname{depth}(x_i) + 1$ comparisons. An unsuccessful search ends in the gap leaf and compares with every node *above* it: depth of the gap comparisons. For example, a search for $x = x_3$ in the tree below needs $2 = 1 + \operatorname{depth}(x_3)$ comparisons, and a search for some $x_2 < x < x_3$ needs 3, the depth of the leaf (x_2, x_3).

The p_i, q_i can be absolute counts, *relative frequencies* (divided by $\sum p_i + \sum q_i$), or *probabilities*. In the last case we minimize the *expected* search cost.

Example ($n = 3$). Frequencies $q_0 = 5$, $p_1 = 2$, $q_1 = 0$, $p_2 = 4$, $q_2 = 7$, $p_3 = 0$, $q_3 = 2$.



cost 44



cost 36 (optimal)

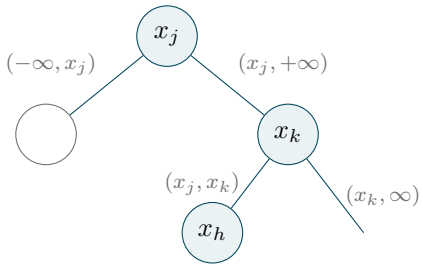
For the left tree (the one of the lecture), the number of comparisons per search and the cost are:

	$(-\infty, x_1)$	x_1	(x_1, x_2)	x_2	(x_2, x_3)	x_3	(x_3, ∞)	
frequency	$q_0 = 5$	$p_1 = 2$	$q_1 = 0$	$p_2 = 4$	$q_2 = 7$	$p_3 = 0$	$q_3 = 2$	
comparisons	1	$0 + 1$	3	$2 + 1$	3	$1 + 1$	2	
product	5	2	0	12	21	0	4	$\Sigma = 44$

The optimal tree has x_2 at the root and costs only 36; we compute it below.

3.6.2 Step 1: Subproblems

Every subtree has an associated **interval** (x_i, x_j): the values x in that interval are exactly the ones whose search leads into that subtree. With root x_j, for example, the left subtree belongs to $(-\infty, x_j)$ and the right one to $(x_j, +\infty)$; if the right subtree has root x_k, its subtrees belong to (x_j, x_k) and $(x_k, +\infty)$, and so on.

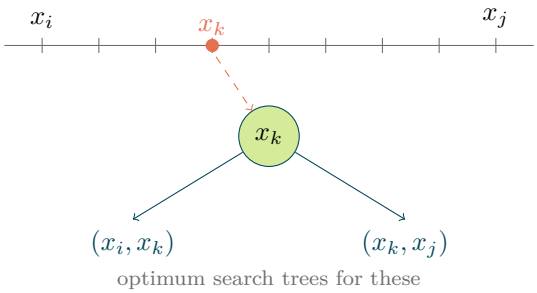


Definition 3.6 (Subproblem P_{ij}). For $0 \le i < j \le n + 1$: P_{ij} = cost of an optimum subtree for the interval (x_i, x_j), that is, for the keys $x_{i+1}, \ldots, x_{j-1}$ and the gaps $(x_i, x_{i+1}), \ldots, (x_{j-1}, x_j)$. Depths are counted from the root of this subtree.

The original problem is $P_{0,n+1}$, the interval $(-\infty, +\infty)$.

3.6.3 Step 2: The Recursion

If we pick x_k $(i < k < j)$ as the root of the subtree for (x_i, x_j), its two subtrees belong to the intervals (x_i, x_k) and (x_k, x_j), and both must be *optimum* search trees for these intervals (otherwise we could replace them by better ones).



The comparison with the root x_k is made by *every* search that enters the subtree, so it adds the total frequency of the interval:

$$w_{ij} := q_i + p_{i+1} + q_{i+1} + \cdots + p_{j-1} + q_{j-1}.$$

Below the root, every search costs exactly one comparison more than inside the subtrees P_{ik} and P_{kj}, and this extra comparison is the one with x_k. Hence

$$\boxed{P_{ij} = \min\{\, P_{ik} + P_{kj} \mid i < k < j \,\} + w_{ij}, \qquad P_{i,i+1} = 0}$$

$P_{i,i+1} = 0$, because there is no key between x_i and x_{i+1}: the subtree is a single gap leaf and needs no comparison of its own.

3.6.4 Step 3: The DP Table

The subproblems are solved in order of increasing interval length $j - i$. For the example above ($n = 3$, so $0 \le i < j \le 4$) the interval weights are $w_{02} = 7$, $w_{13} = 11$, $w_{24} = 9$, $w_{03} = 18$, $w_{14} = 13$, $w_{04} = 20$. Each entry shows "best $P_{ik} + P_{kj}$" $+ w_{ij}$:

$i \backslash j$	1	2	3	4
0	0	$0 + 7 = 7$	$7 + 18 = 25$	$16 + 20 = \mathbf{36}$
1		0	$0 + 11 = 11$	$9 + 13 = 22$
2			0	$0 + 9 = 9$
3				0

For example

$$P_{03} = \min\{P_{01} + P_{13},\ P_{02} + P_{23}\} + 18 = \min\{0 + 11,\ 7 + 0\} + 18 = 25,$$

$$P_{04} = \min\{P_{01} + P_{14},\ P_{02} + P_{24},\ P_{03} + P_{34}\} + 20 = \min\{22,\ 16,\ 25\} + 20 = 36.$$

The minimum for P_{04} is attained at $k = 2$, so x_2 is the root; in P_{02} the only choice is x_1, in P_{24} it is x_3. This is the right tree of the example above. To recover the tree we store, as in the knapsack problem, the minimizing k for every entry.

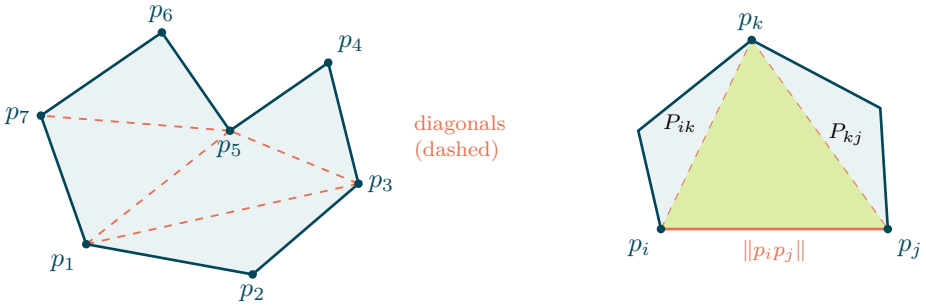
Theorem 3.7 (Optimal binary search tree by DP). The optimal binary search tree for n keys can be computed in $O(n^3)$ time and $O(n^2)$ space.

Proof. There are $O(n^2)$ subproblems P_{ij}. Each one takes the minimum over $O(n)$ choices of k; the weights w_{ij} are computed in $O(1)$ each as $w_{ij} = w_{i,j-1} + p_{j-1} + q_{j-1}$. □

Remark (Knuth's speed-up). One can show that the best root moves monotonically: $\text{root}(i, j-1) \le \text{root}(i,j) \le \text{root}(i+1, j)$. Searching k only in this range gives $O(n^2)$ total time. (Not part of the lecture.)

3.7 Polygon Triangulation

Definition 3.8 (Minimum weight triangulation). Given a polygon $p_1 p_2 \dots p_n$ with n vertices. Partition it into triangles by inserting *diagonals* between vertices (segments inside the polygon that do not cross each other), such that the total length of the diagonals is as small as possible.



Subproblems. For $1 \le i < j \le n$: P_{ij} = optimal triangulation of the subpolygon $p_i, p_{i+1}, \dots, p_j$, closed by the segment $p_j p_i$, *plus* the length $\|p_i p_j\|$ of this closing segment. If $p_i p_j$ is not inside the polygon (not a legal diagonal), then $P_{ij} = \infty$.

Recursion. In a triangulation of the subpolygon, the segment $p_i p_j$ lies in exactly one triangle $p_i p_k p_j$ with $i < k < j$. This triangle splits the subpolygon into the two subpolygons $p_i \dots p_k$ and $p_k \dots p_j$, which must be triangulated optimally:

$$P_{ij} = \begin{cases} \min\{\, P_{ik} + P_{kj} \;:\; i < k < j \,\} + \|p_i p_j\| & \text{if } p_i p_j \text{ is a legal diagonal,} \\ \infty & \text{otherwise;} \end{cases} \qquad P_{i,i+1} = 0$$

$P_{i,i+1} = 0$, since $p_i p_{i+1}$ is a side of the polygon and not a diagonal: it costs nothing.

Overall. The whole polygon is P_{1n}. But $p_1 p_n$ is a side, not a diagonal, so its length must not be counted:

$$\text{OPT} = P_{1n} - \|p_1 p_n\|.$$

Runtime.

- $O(n^2)$ subproblems,
- $O(n)$ choices of k per subproblem,

- $O(n)$ extra overhead per pair (i, j) to test whether $p_i p_j$ is a legal diagonal (intersection test with all n sides; can be done once for all pairs beforehand).

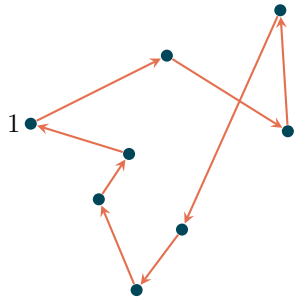
In total $O(n^3)$, again by solving the subproblems in order of increasing $j - i$. The structure is the same as for optimal search trees: an interval (i, j) is split at some k in between.

D DP over Subsets: Traveling Salesman

Lecture 06.11.2026 ▪ Hamiltonian cycles, Held–Karp in $O(2^n n^2)$

3.8 The Traveling Salesman Problem

Definition 3.9 (Traveling salesman problem, TSP). Given a directed graph on the vertices $V = \{1, \dots, n\}$ with arc costs $c_{ij} \in \mathbb{R} \cup \{\infty\}$. Find a *Hamiltonian cycle* (a closed path that visits every vertex exactly once and returns to its start), also called a *tour*, of minimum total length.

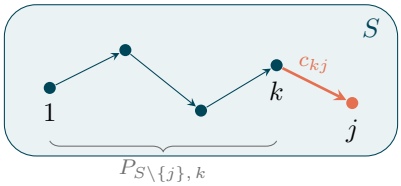


A tour: the salesman visits every city exactly once and returns home, with the shortest total distance.

Brute force. Since a tour is a cycle, we may always start at vertex 1. The remaining $n - 1$ vertices can be visited in any order, so there are $(n - 1)!$ possible solutions.

Idea: build the path incrementally. When we extend a path that starts at 1, we do not need to remember *how* we got to the current point. The only relevant information is the **set S of already visited vertices** plus the **last vertex** so far.

Definition 3.10 (Subproblem $P_{S,j}$). For $S \subseteq V$ with $1 \in S$ and $j \in S$: $P_{S,j}$ = length of a shortest path that visits every vertex of S exactly once and no other vertices, starts at 1 and ends at j.



Recursion. The vertex before j on the path is some $k \in S \setminus \{1, j\}$; the part up to k is a shortest path through $S \setminus \{j\}$ ending at k:

$$P_{S,j} = \min\{\, P_{S\setminus\{j\},k} + c_{kj} \mid k \in S \setminus \{1, j\} \,\} \quad (|S| \geq 3), \qquad P_{\{1,j\},j} = c_{1j} \text{ for all } j \neq 1.$$

Overall. Close the tour by the arc back to 1:

$$\mathrm{OPT} = \min\{\, P_{V,j} + c_{j1} \mid 2 \leq j \leq n \,\}.$$

Order. We process the subproblems in order of increasing $|S|$; then $P_{S\setminus\{j\},k}$ is always known when we need it.

Theorem 3.11 (TSP by DP, Held–Karp). The traveling salesman problem on n vertices can be solved in $O(2^n n^2)$ time and $O(2^n n)$ space.

Proof. There are $O(2^n n)$ subproblems (at most 2^{n-1} sets $S \ni 1$, times n end vertices j). Each takes $O(n)$ time for the minimum over k. The table has one entry per subproblem. □

Remark (comparison with brute force). $2^n n^2$ is exponential, but much smaller than $(n-1)!$: for $n = 20$ it is about $4 \cdot 10^8$ against $19! \approx 1.2 \cdot 10^{17}$. The price is the exponential space $O(2^n n)$.

Notes on Corrections

From the notes of 02.11.2026:

- "Memorization" → *memoization*.
- Variation: $Q_0(c) = \infty$ for $c > 0$ (notes: $Q_0(i)$, $i > 0$); $Q_i(c) = \infty$ means "value c cannot be reached".
- Divide and conquer: the else branch needs $b \geq a_i$; for $b < a_i$ item i cannot be taken. Initialization $Z_{n/2}(b) = b$ added.
- The recursion for $P_i(b)$ needs both cases (written as a brace in the notes): $b \geq a_i$ with the maximum, $b < a_i$ only $P_{i-1}(b)$.
- The DP table was checked by computer: all entries, $P_5(12) = 21$ and $S = \{1, 2, 3\}$ agree with the lecture; the variation gives the same optimum 21.
- Runtime of the space-saving variant: $O(nB \log n)$ as in the lecture is correct; the finer analysis gives $O(nB + n \log n)$.

From the notes of 06.11.2026:

- Optimal BST, example tree: the right child of the root is x_3, not a second x_2 (the depths $1, 0{+}1, 3, 2{+}1, 3, 1{+}1, 2$ in the table only fit this tree). Its cost is $5{+}2{+}0{+}12{+}21{+}0{+}4 = 44$; the optimal tree (root x_2) costs 36.
- Subproblems P_{ij}: the index range is $0 \leq i < j \leq n+1$ (notes: $i \leq j$); for $i = j$ the interval is empty. Likewise $1 \leq i < j \leq n$ for the polygon.
- The table was checked by computer, and the optimum 36 also by trying all 5 binary search trees on 3 keys.
- Optimal BST: the runtime $O(n^3)$ was not in the notes and is added.
- TSP: the salesman visits *every* city exactly once (notes: "the most number of cities"). In the recursion k must also differ from 1: $k \in S \setminus \{1, j\}$ (notes: $S \setminus \{j\}$), because a path ending at 1 is only defined for $S = \{1\}$.
- The two pictures copied into the notes (a cartoon and a web screenshot about Hamiltonian cycles) are not reproduced; the definition of a Hamiltonian cycle is in Definition 3.9.